Charging the Selector: Peer-Selection Cost and Measured Latency of Sparse Attention in Multi-Agent Communication

Anonymous authors
Paper under double-blind review

Abstract

Sparse attention in multi-agent communication is usually justified by the operations it removes. We charge each peer-selection rule for what deciding costs, and measure on four GPUs, from A100 to Blackwell, when the saving becomes elapsed time. First, a rule that ranks attention scores must compute all of them, so it removes at most half of the forward-pass multiply–accumulates at any sparsity; rules that select before scoring escape this ceiling by as much as their selector is cheap. Second, composed from library operators, score-then-select attention is slower than dense attention at every configuration we measure, by up to 37 times, because compacting the retained peers adds more memory traffic than it removes. A fused kernel that streams the selection is faster than dense attention with one or two retained peers in fp32 and, up to 512 agents, within 15% of parity with four and slower with eight; and from 384 agents upward at least 85% of its time is spent finding the retained set; in bf16 only one retained peer breaks even. On A100, H100 and H200 the streaming kernel wins everywhere only with one retained peer, and in bf16 not at all. Third, once the set is known without scoring, attention over it is up to 16 times faster than dense attention, and the selector decides the outcome: of three selectors, only the cheapest reaches parity, at 512 agents, on best-case timings and on the Blackwell card only. Below 100 agents, where multi-agent methods are usually evaluated, the number of kernels a step launches orders latency, and its operation count does not.

1 Introduction

Cooperative multi-agent systems increasingly exchange learned messages, and attention is a common way of deciding whose message matters (Das et al., 2019; Jiang & Lu, 2018; Jiang et al., 2020).[1] Attention over N agents is pairwise, so its cost grows quadratically, and several methods restrict who attends to whom, citing communication or computational cost among their reasons (Jiang et al., 2020; Sun et al., 2020). The computational argument is an operation count: if each agent aggregates messages from k peers instead of $N-1$, the aggregation shrinks by a factor of about N/k.

This paper asks when that saving becomes elapsed time. Three costs have to be separated, and an operation count conflates them. The first is *what can be removed at all*: a rule that selects peers by their attention scores needs the scores, so every pairwise score has been computed by the time the selection is made, and only the aggregation can shrink. The second is *what the implementation executes*: masking the discarded weights to zero leaves the dense multiplication in place, while gathering the retained values removes it and writes new intermediates instead. The third is *what the hardware spends its time on*: at the agent counts and head widths of this setting, attention is memory-bound, so an implementation that removes arithmetic while adding data movement can become slower. We study centralised inference on a single device, the

[1] Large language model assistants were used in this work to write and test code, to edit the manuscript, and to audit its numbers, derivations and citations against the released artifacts and the cited sources. Every measurement was produced by the released scripts, and the authors take responsibility for all content.

setting in which the methods we cite are trained and evaluated, and not bandwidth-constrained distributed deployment, where reducing transmitted messages is itself the objective.

The object of the paper is that accounting rather than any one mechanism. Most multi-agent methods that restrict communication choose peers before scoring them, from positions, per-agent gates or a separate network (Section 2). *Score-then-select*, which ranks the attention scores themselves, is uncommon in this literature, and the cost argument made for it is a reduction in complexity (Sun et al., 2020), so this paper does not set out to correct a measured result. We treat it as the limiting case of a selector-cost accounting that covers every rule, and measure each case with a protocol built for kernels that run for microseconds.

Findings. For *score-then-select* rules, which form every score before keeping the k largest, at most half of the forward-pass attention multiply–accumulates are removable at any sparsity, a limit noted before in a memory-access model (Chen et al., 2023). Composed from library operators, such a rule is slower than dense attention at every agent count and sparsity we measure, the gathered form by up to thirty-seven times, because of memory traffic. Fused kernels that hold each query's score row on chip are faster only with one retained peer; a kernel that streams the selection through a running candidate set is faster with one or two in fp32 and close to parity with four, but in every kernel most of the time from 384 agents upward is spent finding the retained set; in bf16, which accelerates the products and not the selection, only a single retained peer breaks even. Rules that *select before scoring* escape the bound: attention over a pre-selected set is up to sixteen times faster than dense attention, and the selector then decides the outcome, with only the cheapest of three reaching parity, at 512 agents. Below a hundred agents, where the methods we cite are evaluated, none of this governs latency: the number of kernels a communication step launches does.

Contributions.

- A selector-cost accounting (Section 3) that charges every peer-selection rule for deciding what to keep and sorts mechanisms into four asymptotic classes (Proposition 2, Corollary 1). Its limiting case, score-then-select, has a closed-form multiply–accumulate limit for the forward and backward passes (Proposition 1), and a byte criterion, $kd \leq N$, screens compaction before implementation.
- A measurement protocol for microsecond-scale attention kernels (Section 4): masked, gathered and fused implementations of one function, verified equivalent, timed interleaved against the best of a vendor library, FlexAttention and a dense kernel searched over the sparse kernels' parameter space, in three numerical formats; a fixed-set comparison that separates finding the retained peers from using them; and a checklist of eight errors that each produced a plausible but spurious result (Section 5.2).
- A characterisation of when sparse attention is faster in this regime: narrowly for score-then-select, and more broadly for selection before scoring when the selector is cheap in absolute terms and the remaining attention is fused (Sections 4.4–4.6).

We release the kernels, the harness and the scripts that regenerate every table and figure and check the quoted numbers. A pre-registered training study (Appendix G) shows that the communicating architecture improves return under the evaluated setup; it is not powered to rank sparsification variants, and no claim in the main text rests on it. The measured envelope is four GPUs across three generations for the headline quantities and one (Blackwell) for the rest, head widths 16 to 512 and at most 512 agents (Section 5).

2 Related Work

Communication sparsity in multi-agent reinforcement learning. CommNet broadcasts mean-pooled hidden states (Sukhbaatar et al., 2016), TarMAC introduced signature-based attention targeting (Das et al., 2019), and MAT casts the joint policy as a sequence model (Wen et al., 2022). Because attention over N agents is pairwise, several of these methods restrict who communicates, and some motivate the restriction by cost; DGN observes that receiving much information "requires high bandwidth and incurs high computational complexity" (Jiang et al., 2020). Corollary 1 sorts these mechanisms by what their selector costs. DGN's nearest neighbours, ATOC's communication field and mean-field interaction fix the retained

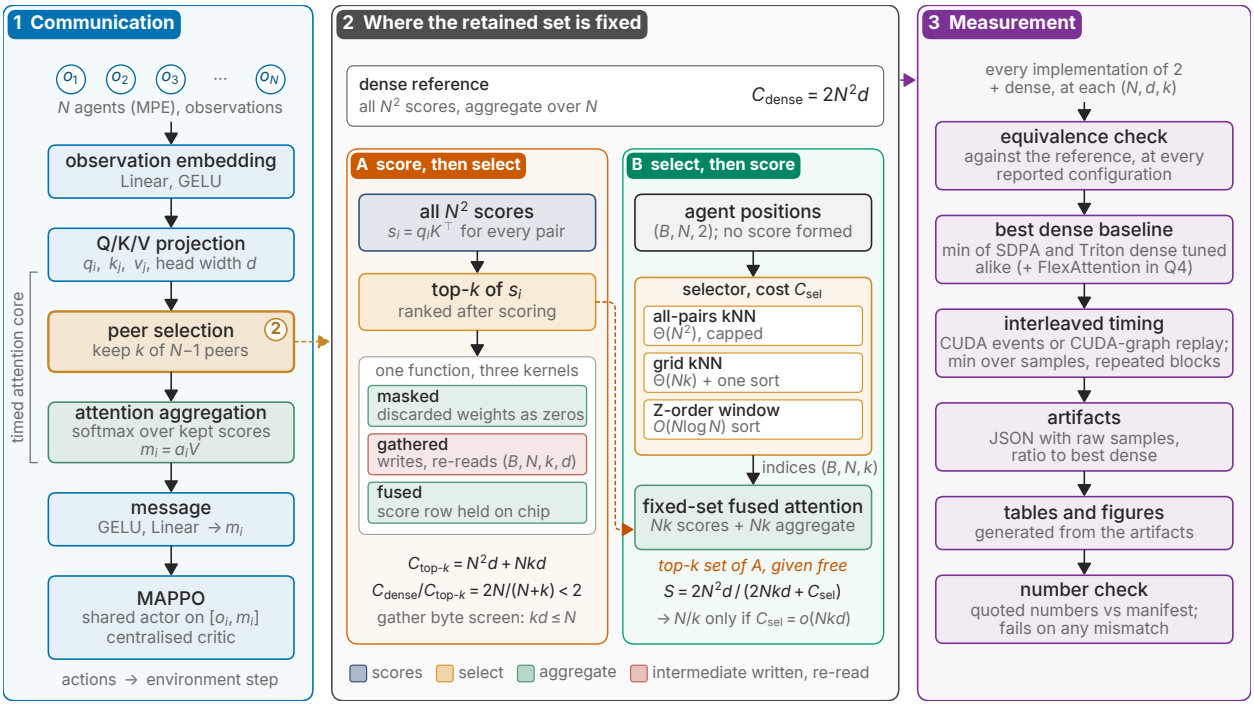


Figure 1: The study at a glance. **(1)** The communication module: observations are embedded and projected to queries, keys and values, a peer-selection step keeps k of the $N-1$ peers, and the aggregated message m_i joins the observation at the shared MAPPO actor; every latency we report is of the attention core bracketed at left. **(2)** Where the retained set is fixed, with the operation counts of Section 3: score-then-select (A) forms all N^2 scores before ranking them and admits three implementations of one function, while selecting before scoring (B) pays a selector cost C_{sel} and then attends over Nk pairs only; the dashed arrow is the fixed-set comparison, which hands the exact top-k set to the same fused kernel at no cost. **(3)** Every implementation is verified against a reference and timed interleaved against the best dense implementation, and every table, figure and quoted number is generated from, or checked against, the resulting artifacts.

set from structure available before any score exists (Jiang et al., 2020; Jiang & Lu, 2018; Yang et al., 2018); they are free if the neighbourhood arrives with the environment and case (b) if it is recomputed from all pairwise positions each step. IC3Net and SchedNet gate or rank agents by a scalar per agent, a top-k over an $O(N)$ vector rather than an $N \times N$ matrix (Singh et al., 2019; Kim et al., 2019). G2ANet and MAGIC run a pairwise hard-attention or graph-construction network before communication attention (Liu et al., 2020b; Niu et al., 2021), so their selector is of the same order as the scores they avoid. Score-then-select is the limiting case: the adaptive sparse communication graph of Sun et al. (2020) applies a sparsemax generalisation to logits already formed for every pair and motivates it partly by reduced complexity, and handshake mechanisms that match a query against every candidate key before selecting (Liu et al., 2020a) are score-then-select in the same sense. Most of these methods are evaluated with two to roughly thirty agents; DGN and ATOC reach about one hundred, and mean-field MARL, which replaces communication with a population average, one thousand.

Efficient and sparse attention. Efficient-attention methods separate along the same axis. Sparse Transformers, Longformer and BigBird fix the interaction pattern in advance, so the score computation itself is sparse and the selector is essentially free (Child et al., 2019; Beltagy et al., 2020; Zaheer et al., 2020). Reformer and Routing Transformer restrict candidates by hashing and online clustering, which are content-dependent but evaluate scores only for candidate pairs, at a selector cost that must be counted (Kitaev et al., 2020; Roy et al., 2021). Performer keeps every interaction and reorders the algebra (Choromanski et al., 2021). Top-k attention in sequence models is the closest analogue of our mechanism: the Explicit

Sparse Transformer keeps the k largest scores per query (Zhao et al., 2019), and Gupta et al. (2021) compute each query's exact top-k over all keys in chunks, with memory efficiency as the stated benefit. That top-k attention cannot approach N/k has been noted before: Chen et al. (2023) bound its speedup by a constant in a memory-access model even with a mask supplied at no cost, and Block-Sparse FlashAttention computes every query–key score to choose which value blocks to skip, skipping about half the work of a pruned block and reporting kernel speedups of up to 1.38× (Ohayon et al., 2026). Sparsemax and α-entmax (Martins & Astudillo, 2016; Peters et al., 2019) produce their zeros after the scores exist, so by our reading they fall under Proposition 1. Sparse mixture-of-experts routing is also score-then-select (Shazeer et al., 2017; Fedus et al., 2022), but the router is small relative to the expert computation it gates, so the analogue of our score term is minor and the saving is real. Whether score-then-select pays depends on the ratio between the cost of scoring every candidate and the cost of the work skipped, which is one in single-head attention.

Hardware-aware execution. That operation counts do not determine runtime is established in general terms (Dehghani et al., 2022), and transformer workloads are known to be dominated by data movement rather than arithmetic (Ivanov et al., 2021); the roofline model formalises when arithmetic ceases to govern runtime (Williams et al., 2009). FlashAttention and its successors reduce memory traffic without reducing the operation count (Dao et al., 2022; Dao, 2024; Shah et al., 2024; Rabe & Staats, 2021), and FlexAttention compiles attention variants expressed as score modifications and block masks into fused kernels (Dong et al., 2025). Native Sparse Attention and MiniMax Sparse Attention show where sparse attention does pay: blockwise selection amortises the selector over blocks, preserves matrix-multiply shapes inside selected tiles, and operates at 10^4 to 10^6 tokens, where the skipped work is large (Yuan et al., 2025; Lai et al., 2026). Relative to this literature our contribution is specific: a selector-cost accounting that separates mechanisms before measurement, a byte criterion for per-pair compaction, and measurements at tens to hundreds of agents and narrow heads, where neither block amortisation nor large skipped work is available. We follow the benchmarking recommendations of Hoefler & Belli (2015) where they apply, reporting absolute latencies beside every ratio together with the statistic and its run-to-run dispersion, and our kernels are written in Triton (Tillet et al., 2019).

3 A Cost Model for Peer Selection

3.1 Setting and implementations

Consider one round of scaled dot-product attention over N agents, per batch element and head, at head width d. Agent i forms a query q_i; every agent contributes a key and a value; self-attention is masked. Write $s_i = q_i K^\top \in \mathbb{R}^N$ for the score vector and $a_i = \mathrm{softmax}(s_i)$, so that dense attention computes $m_i = a_i V$. A top-k variant keeps the k largest entries of a_i and renormalises over them. We call a rule that forms the complete score vector and then keeps its k largest entries *score-then-select*. Every analytic claim below concerns that class, not content-dependent selection in general, and Section 3.3 draws the boundary.

Attention has two matrix products of equal size: one produces the pairwise scores and the other aggregates the values. Restricting the retained set shrinks the second, because only k value rows are combined, and cannot shrink the first, because the scores are what the selection reads. The same top-k function also admits several implementations (Figure 2), and whether the saving is executed depends on which one runs.

Masked top-k multiplies the dense weights by a $\{0, 1\}$ mask and renormalises. The discarded entries are materialised as zeros and multiplied, so the executed aggregation is that of dense attention.

Gathered top-k takes the top-k of the raw scores, applies a softmax over those k values, gathers the corresponding value rows into a (B, N, k, d) tensor and contracts it. The aggregation genuinely shrinks.

Fused top-k computes the same function in one kernel, holding the score row on chip, selecting within it and accumulating the weighted values without writing a score matrix, an index tensor or a gathered tensor to memory (Section 4.4).

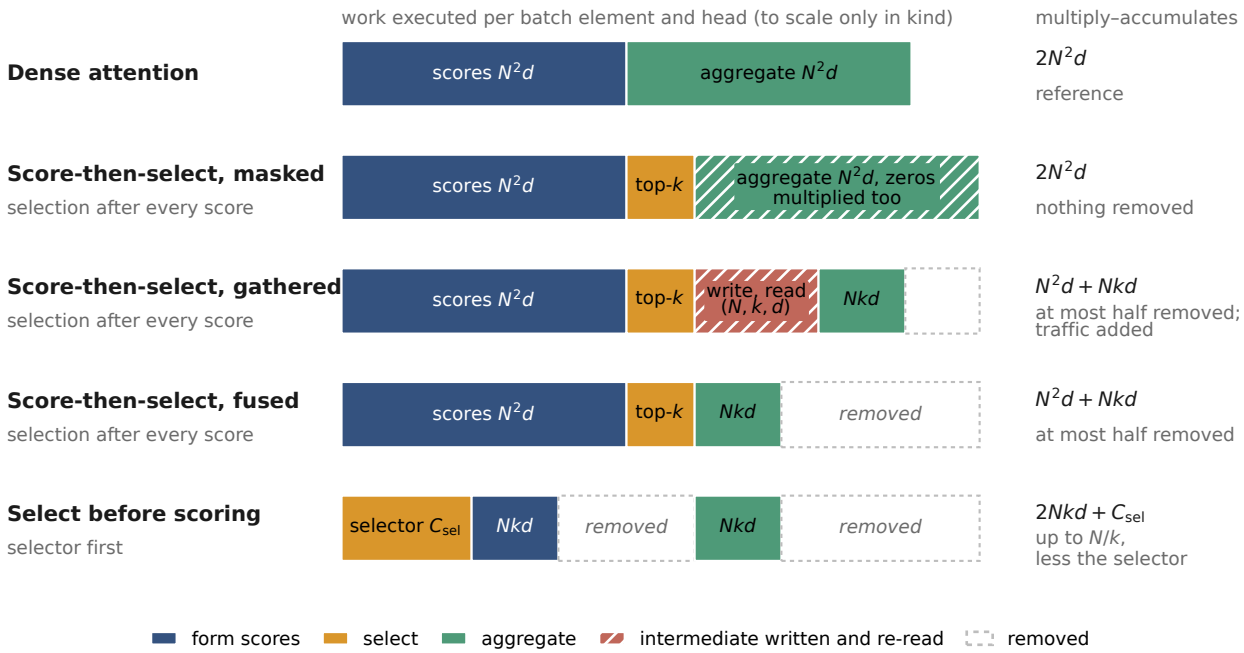


Figure 2: What each kind of peer selection removes and what it still pays for, per batch element and head. Box colour gives the role of a stage; widths indicate the kind of term, not measured time. Every score-then-select variant forms all N^2 scores before it can rank them, so at most the aggregation can shrink: the masked variant multiplies the discarded weights as zeros, the gathered one removes them but writes and re-reads a compacted tensor, and only the fused one removes them without adding traffic. Selecting before scoring removes score and aggregation terms alike, at the price of the selector. The threshold kernel of Section 4.4 is fused but keeps the dense aggregation.

Gathered and masked top-k compute the same function, since softmax is monotone within a row and a softmax over the k retained scores equals the dense softmax restricted to them and renormalised; all implementations we time agree to floating-point tolerance (Appendix F).

3.2 An operation-count ceiling for score-then-select

We count multiply–accumulates (MACs) of the two attention products and exclude the input and output projections shared by every implementation:

$$C_{\text{dense}} = \underbrace{N^2 d}_{\text{scores}} + \underbrace{N^2 d}_{\text{aggregate}}, \qquad C_{\text{top-}k} = \underbrace{N^2 d}_{\text{all scores}} + \underbrace{Nkd}_{\text{aggregate } k}. \tag{1}$$

Proposition 1 (Score-then-select ceiling)**.** *Let the retained set be fixed by a score-then-select rule, one whose kept indices are determined only after the complete score vector of each query has been formed, and let keys and values have equal width. Then, for the forward pass,*

$$\frac{C_{dense}}{C_{top\text{-}k}} = \frac{2N}{N+k} < 2 \qquad \textit{for all } k \geq 1, \tag{2}$$

with supremum 2 *approached only as* $k/N \to 0$.

Why it matters. The saving usually quoted for top-k communication, $1 - k/(N-1)$, describes the aggregation term alone. Read as the saving of the mechanism it overstates the achievable reduction by exactly a factor of two (under the masked-diagonal count the removable fraction is $\frac{1}{2}(1-k/(N-1))$), before any question of whether the reduction is realised in time. The bound depends only on *when* the retained set

is fixed: a rule that must read every score pays for every score. The observation is not new in kind: Chen et al. (2023) bound the speedup of top-k attention by a constant in a memory-access model even when the mask is supplied at no cost. Proposition 1 states the limit as a closed-form operation count, extends it to training, and serves as the limiting case of the selector accounting below.

The proof, the treatment of the masked diagonal and seven assumptions (single round, full-score selection, per head, projections excluded, selection cost excluded, equal key and value widths, element-wise work excluded) are in Appendix A. Relaxing the single-round, per-head, projection or selection-cost assumption leaves the ratio unchanged or moves it closer to one; values wider than keys raise the ceiling to $1 + d_v/d_k$, and charging the softmax can lift it slightly above two. The ceiling is a statement about the forward pass. In training, exact selection is piecewise constant in the scores, so a backward written for it needs the gradient products only over the retained pairs, and the analogous bound is $6N/(N + 5k) < 6$; the autograd backward we time forms dense query and key gradients and does not reach it (Appendix A). Every timing claim in this paper concerns the forward pass unless stated otherwise.

3.3 Selection before scoring

Mechanisms that are content-dependent but do not read the complete score vector lie outside Proposition 1: nearest-neighbour retrieval, clustering, routing, hashing and fixed patterns can identify candidates without forming every score (Kitaev et al., 2020; Roy et al., 2021; Yuan et al., 2025). What they pay instead is the cost of deciding, and we charge it explicitly.

Proposition 2 (Selection before scoring)**.** *Suppose the retained set is determined without forming the complete score vector, at total selector cost $C_{\mathrm{sel}}(N, k, d)$, and that scores are then evaluated only for the Nk retained pairs. Then*

$$S \;=\; \frac{C_{dense}}{C_{\mathrm{pre}}} \;=\; \frac{2N^2 d}{2Nkd + C_{\mathrm{sel}}}. \tag{3}$$

(i) Along any sequence with $N \to \infty$, k and d allowed to depend on N: if $C_{\mathrm{sel}} = o(Nkd)$, then $S/(N/k) \to 1$; equality with N/k holds only for $C_{\mathrm{sel}} = 0$. (ii) If $C_{\mathrm{sel}} = o(N^2 d)$ and $k/N \to 0$, then S diverges, though not in general at the rate N/k.

A score-then-select rule is the limiting case. It has formed every score while selecting, so written in this form its selector cost is at least the scores of the discarded pairs, $C_{\mathrm{sel}} \geq N^2 d - Nkd$, and substituting recovers Proposition 1. Read together, the two propositions place every peer-selection mechanism on one axis — the cost of deciding what to keep — and the position on that axis bounds what the mechanism can save.

Corollary 1 (What a selector costs, and what it can buy)**.** *Count C_{sel} in MAC-equivalents of two scalar operations each, so that a comparison or integer operation is half a unit. Fix k and d and let $N \to \infty$.* (a) Full-score selection*, $C_{\mathrm{sel}} \geq N^2 d - Nkd$: $S < 2$.* (b) All-pairs selection in a feature space of dimension p*, $C_{\mathrm{sel}} = \gamma N^2 p$: $S \to 2d/(\gamma p)$, a constant.* (c) Sub-quadratic selection*, $C_{\mathrm{sel}} = \Theta(N \log N)$: $S = 2Nd/(2kd + \Theta(\log N)) = \Theta(N/\log N)$, unbounded but a vanishing fraction of N/k.* (d) Linear selection*, $C_{\mathrm{sel}} = cNkd$ for a constant $c > 0$: $S = 2N/((2 + c)k)$ exactly; and $S \sim N/k$ when $C_{\mathrm{sel}} = o(Nkd)$, as for a fixed or precomputed pattern.*

Why it matters. Only a free selector delivers the N/k that operation-count arguments for sparse communication invoke, and full-score selection is bounded by two at every N. The intermediate cases are where multi-agent methods sit, and they differ: a selector that ranks all pairs in a feature space of per-pair cost γp is capped at $2d/(\gamma p)$ — above two only when $\gamma p < d$ — while a sub-quadratic one is unbounded. The lightning indexer of DeepSeek Sparse Attention is a large-scale instance of case (b): it scores every query–key pair with a cheaper function, and its authors note that it remains quadratic (DeepSeek-AI, 2025). The asymptotic classes do not depend on the unit convention; the constants in the closed forms of Section 4.6 do. Proofs are in Appendix A.

3.4 Why the count need not govern the time

For memory-bound kernels, time tracks bytes moved rather than arithmetic performed (Williams et al., 2009). At head width 16, the two composed paths in this study have an arithmetic intensity below every machine balance we measure on the same device (Appendix A.5), so their accounting is of bytes; fused kernels, which never write the score matrix, are not covered by this argument. Gathering does not remove memory traffic; it relocates it. Instead of multiplying $N - k$ zeros, the gathered path writes and re-reads a compacted (B, N, k, d) tensor and the indices that build it.

Byte criterion. Count each tensor read or written once, in fp32 with int64 indices. Replacing the dense softmax and aggregation by selection, a softmax over k, the gather and the compacted contraction changes traffic by $2Nkd + 8Nk - 2N^2$ elements per batch element: the compacted tensor written and re-read costs $2kd$ per query where the dense weight row written and re-read cost $2N$. Gathering is therefore byte-neutral only if $k(d + 4) \leq N$, to leading order $kd \leq N$. At d=16 and N=512 the leading-order form admits $k \leq 32$ ($k \leq 25$ exactly); at d=128, $k \leq 4$ ($k \leq 3$). Unlike the operation count, this criterion depends on d and can be checked before any code is written. With int32 indices the index term halves and the exact condition becomes $k(d + 2) \leq N$; in a narrower value format the index term grows relative to the rest. The leading-order form is unchanged in every case, and the composed path cannot use int32 in any event, since PyTorch's gather takes int64 indices. It is a screen under a simplified accounting that ignores caches and fusion, not a model of GPU time; hardware counters on two GPUs agree with it (Appendix B.5). Section 4 tests the ceiling, the selector classes and the byte criterion against measurements.

4 Experiments

The experiments answer six questions in turn. Does score-then-select reduce the computation that is executed (Q1, Section 4.2)? Does the reduced computation become reduced latency (Q2, Section 4.3)? When can a fused score-then-select kernel be faster than dense attention, and where does its time go (Q3, Section 4.4)? Does the answer depend on numerical precision (Q4, Section 4.5)? What changes when peers are selected before scoring (Q5, Section 4.6)? And what governs latency at the agent counts of current methods (Q6, Section 4.7)?

4.1 Setup

We time the attention core — from queries, keys and values to the message tensor — because the input and output projections are shared by every implementation and dilute the quantity in question. Unless stated otherwise the configuration is fp32, head width d=16, batch 256, on one NVIDIA Blackwell GPU (sm_120) under PyTorch 2.11, CUDA 12.8 and Triton 3.6. The dense baseline is the fastest of PyTorch's fused scaled-dot-product path and a streaming dense kernel we wrote in Triton and tuned over the same parameter space and admissibility filter as our sparse kernels; Q4 adds FlexAttention (Dong et al., 2025). Every ratio in this paper is against that best dense implementation, and a ratio below one means the sparse path is faster.

Timing uses on-device CUDA events with explicit synchronisation. The device was shared with other tenants and clocks could not be locked, so implementations are timed *interleaved*, repetition by repetition, within one sweep per agent count; interference then lands on all of them together and their ratios stay paired. The primary statistic is the minimum over samples, an estimate of the uncontended cost; the whole block is repeated three times and we report where the ratio of medians changes a conclusion. Recomputing every comparison from medians of the same samples changes which implementation is faster at one of 24 configurations in the main sweep, and reverses the end-to-end pre-score result of Q5 in one of its two sweeps. The first is the fused kernel at one retained peer, 384 agents and d=16, where one of the three runs carried heavy contention; re-timed under CUDA-graph replay in three further sessions it is faster than dense on minima and medians alike (0.92 on both, every one of nine blocks between 0.92 and 0.93; Table 8). The second is stated in Q5. Below 192 agents a fixed per-kernel cost is a large share of every measurement, so Q1–Q5 are read from 192 agents upward and the smaller counts are measured separately under CUDA-graph

Table 1: Forward latency (ms, minimum over interleaved samples) of one function computed six ways, at a quarter of peers retained. The composed sparse path is slower than every dense implementation at every agent count, and the gap widens with N; the fused kernels change the picture only at small k (Section 4.4). Rows below N=192 are dominated by per-kernel cost and are read in Section 4.7.

		dense				**top-k**			
N	k	unfused	fused (library)	ours	ours (tuned)	gathered (composed)	fused gather	fused mask	best sparse ÷ best dense
12	2	0.012	0.013	0.004	0.004	0.022	0.004	0.006	0.99
24	4	0.015	0.011	0.004	0.004	0.028	0.006	0.016	1.37
48	8	0.020	0.015	0.008	0.008	0.048	0.012	0.024	1.54
96	16	0.045	0.033	0.018	0.014	0.120	0.049	0.039	2.80
192	32	0.136	0.072	0.036	0.034	0.424	0.387	2.305	11.29
384	64	1.082	0.220	0.119	0.108	2.405	2.750	2.025	18.70
512	128	1.899	0.349	0.188	0.172	6.435	7.214	2.662	15.44

replay in Q6. Every implementation is checked against a reference at every configuration whose latency is reported, and all agree to fp32 rounding except at exact ties at the k-th score (Appendix F). Appendix B.1 gives the full protocol, including the tiling search, and Appendix C the errors it guards against. Appendix B.5 repeats the headline measurements on an A100, an H100 and an H200, each allocated exclusively and with every tiling searched again on the device, and reports where they differ from the Blackwell card.

4.2 Q1: Is the reduction executed?

Only in some implementations, and the count cannot tell which. PyTorch's FLOP counter, attached to the running program, confirms that the gathered implementation executes the reduction Proposition 1 permits. The masked implementation multiplies the discarded entries as zeros and so executes dense arithmetic plus selection, while a count based on the retained fraction credits it with a 75% saving at N=512, k=127.

4.3 Q2: Does the reduction become time?

Not in this regime. Composed from general-purpose operators, the gathered path executes less arithmetic than dense attention and is slower than every dense implementation at every agent count we measure, by up to a factor of thirty-seven at 512 agents and a quarter of peers retained (Table 1).

The byte criterion of Section 3.4 accounts for this: at d=16 and N=512 the quarter-of-peers operating point is four times past its leading-order bound. Timed in isolation, the sparse contraction and the dense multiply it replaces both run near the bandwidth the device attains on a large copy, so neither is on a worse hardware path; the sparse one is slower because it moves about four times the bytes. The largest single stage of the composed path is selection, which runs at under a tenth of attainable bandwidth (Appendix B.6).

These are attention-stage measurements, and two wider views shrink them. The ordering survives the backward pass with a narrower margin — at 512 agents the gathered path is 18.4 times the fused dense path on the forward pass and 6.2 times on forward and backward, since selection has almost no adjoint — and in a complete MAPPO iteration at six to twenty-four agents the environment and per-step action selection take 78 to 96% of the time and all implementations are within 2% of one another (Appendix B.7). Neither a speedup nor a slowdown of the attention stage is an end-to-end speedup or slowdown in that pipeline.

4.4 Q3: When is a fused score-then-select kernel faster, and where does its time go?

The composed path spends most of its time moving intermediates that a fused kernel need not create. We wrote fused kernels that hold the score row on chip and never write a score matrix, an index tensor or a gathered value tensor to memory. At 512 agents retaining a single peer the composed sparse path

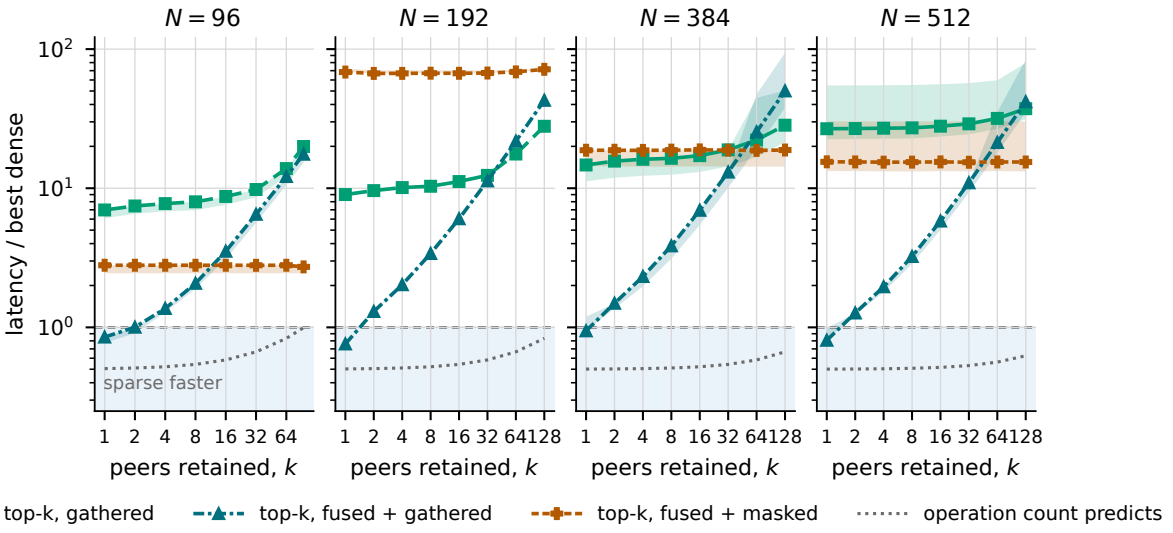


Figure 3: Latency divided by the fastest dense implementation, against the number of peers retained. The dotted line is what an operation count predicts for the gathered path, $(N + k)/2N$, which lies below parity everywhere and so predicts a saving at every sparsity. Shading marks the region in which sparsity is actually faster; only the fused gathered kernel enters it, and only at the smallest k. Lines are ratios of minima; bands span the ratios of the three runs' medians, and so show how far each point moves between runs and under a change of estimator. The N=96 panel is dominated by per-kernel cost; Section 4.7 covers that range.

costs 4.62 ms and the fused one 0.139 ms, a factor of 33 attributable to the implementation and not to the algorithm, and the fused path is then faster than the best dense implementation (Figure 3).

Matched authorship is a weaker condition than matched efficiency, so we searched both kernels over one parameter space — query tile, key tile or head chunk, warp count and pipeline depth — under one shared-memory admissibility filter, and report the fraction of the device's matrix-multiply rate each reaches (Table 9, Appendix B.2). At head widths 16 and 32 the fused sparse kernel is the *less* efficient of the two at every agent count — 21 to 33% of the device's rate against 30 to 47% for dense — and it is faster at one retained peer only because it performs about half the arithmetic. The search is not equal: the sparse kernel must hold an N-wide score tile, so above 192 agents the filter admits three sparse configurations against 28 to 106 dense ones. The sparse figures are therefore lower bounds, the negative entries at head width 64 and above are weakly supported, and the positive entry is conservative. We call the sparse kernel the best implementation we found under these constraints, not an optimal one.

For kernels that hold the score row resident, the advantage is narrow. It appears at one retained peer, and at two only at d=32, N=512; the k=1 case is degenerate as attention, since a softmax over one element is identically one and the mechanism reduces to copying the highest-scoring peer's value. At four retained peers the fused kernel is 1.3 to 2.3 times slower than dense, and at a quarter of peers more than an order of magnitude slower. Three ways of selecting inside the kernel — repeated argmax, bisection on the score's bit pattern, and one sort of score–index pairs — trade a cost linear in k for dense aggregation or for a sort that costs five to sixteen times dense attention, and none changes the outcome (Appendix B.2).

To see where the fused kernel's time goes, we held the retained set fixed and changed only how it is found (Table 2). Given the exact top-k indices for free, a fused kernel that reads one key and one value row per retained peer costs 0.06 to 0.11 times dense attention at 512 agents and $k \leq 4$; the fused score-then-select kernel, attending over the same peers, costs 0.77 to 1.93 times. From 384 agents upward, at least 90% of the fused kernel's device time is the cost of finding the set, which is the term Proposition 1 says cannot be removed. Finding the same set with library operators instead, a score product followed by `torch.topk`, costs 8 to 27 times dense attention on its own at d=16. This sweep ran on the other card of the host from the main sweep, and the fused kernel's ratios reproduce it (0.77, 1.27 and 1.93 against 0.81, 1.27 and 1.96

Table 2: One retained set, reached two ways: found by scoring inside the fused score-then-select kernel, or handed to a fused kernel that attends over a given set. Both compute exact top-k attention over the same peers (maximum deviation from the reference 2.4×10^{-6}). Ratios of minima against the best dense implementation, under CUDA-graph replay so that all are device times. For the score-then-select kernel the median of the per-repetition paired ratios is within 3.3% of the ratio of minima, with a 95% bootstrap interval within ±0.8%; for the fixed-set kernel, which runs for a few microseconds, within 14% and ±3.4%. The last column is the share of the fused kernel's time not accounted for by attending over the set, across the five k.

		found by scoring / given for free, ÷ best dense					**finding**
d	N	k=1	k=2	k=4	k=16	k=32	**the set**
16	192	0.79 / 0.24	1.29 / 0.24	2.01 / 0.30	5.87 / 0.59	10.98 / 1.02	69–91%
	384	0.91 / 0.09	1.48 / 0.11	2.26 / 0.15	6.84 / 0.37	12.95 / 0.71	90–95%
	512	0.77 / 0.06	1.27 / 0.07	1.93 / 0.11	5.80 / 0.32	11.00 / 0.61	92–94%
32	192	0.74 / 0.16	1.09 / 0.20	1.51 / 0.26	3.93 / 0.45	7.18 / 0.79	78–89%
	384	0.82 / 0.07	1.16 / 0.08	1.60 / 0.12	4.14 / 0.32	7.56 / 0.60	92–93%
	512	0.67 / 0.05	0.95 / 0.06	1.30 / 0.09	3.32 / 0.25	6.05 / 0.48	92–94%

Table 3: A streaming exact top-k kernel, which walks the keys in tiles and keeps a running candidate set per query instead of the score row: latency divided by the best dense implementation (ratios of minima under CUDA-graph replay, fp32, batch 256), beside the resident-row kernel at four retained peers. The streaming kernel retains exactly the peers `torch.topk` does in every one of 34 million query rows checked. The last column is the share of the streaming kernel's time not accounted for by attending over the set, across k, as in Table 2. Rows at 1024 agents are beyond the resident kernels' cap.

		streaming, ÷ best dense				**resident**	**finding**
d	N	k=1	k=2	k=4	k=8	k=4	**the set**
16	192	0.62	0.81	1.15	1.76	2.05	72–80%
	384	0.57	0.70	1.13	1.66	2.29	87–89%
	512	0.57	0.69	1.12	1.52	1.92	88–90%
	1024	0.54	0.67	1.03	1.29	—	—
32	192	0.63	0.84	1.06	1.41	1.51	77–80%
	384	0.50	0.63	0.98	1.25	1.58	85–88%
	512	0.50	0.60	0.91	1.11	1.26	87–91%
	1024	0.46	0.56	0.78	0.92	—	—

at d=16, N=512, $k \in \{1, 2, 4\}$). What is fundamental to exact score-then-select is that all N scores are *computed*; holding them *resident* is a property of the kernels above, and it is what restricts their tiling.

Streaming the selection. We therefore wrote a streaming exact top-k kernel in the manner of the chunked selection of Gupta et al. (2021): each program takes a tile of queries, walks the keys in tiles as a dense streaming kernel does, and merges each score tile into a running set of k candidates held in registers, packing each score with its index so that one integer comparison orders both, as `torch.topk` does on ties. Nothing N-wide stays on chip, so the kernel admits the dense kernel's tilings and has no cap on N. It changes the picture for small k (Table 3). At one and two retained peers it is faster than the best dense implementation at every configuration measured, at 0.50 to 0.84 times its latency; at four it is within about 15% of parity (0.91 to 1.15 up to 512 agents), where the resident kernel is 1.3 to 2.3 times slower; and at eight it is slower again up to 512 agents. Two retained peers is a genuine attention, so the advantage of score-then-select over dense attention in fp32 is not confined to the degenerate case. It remains bounded: the floor is the score product, about half of dense attention, and from 384 agents upward 85 to 91% of the streaming kernel's time is still finding the set. The window is device-dependent. On the A100, H100 and H200 the streaming

Table 4: The fused crossover in three numerical formats. Each hand-written kernel is searched separately in each format; the best dense implementation is the fastest of the vendor library, FlexAttention (*flex*) and our tuned kernel (*ours*). Ratios are of minima, with the ratio of medians at one retained peer beside it. A dash marks a configuration in which TF32 rounding changed the retained peer in more than 0.1% of query rows. The last column is top-k through FlexAttention with a block mask built from the retained set, including the per-call mask construction.

			best dense		**fused top-k ÷ best dense**				**Flex**
d	N	format	from	ms	k=1	k=1 (med.)	k=2	k=4	k=1
16	192	fp32	ours	0.0325	0.76	0.73	1.25	2.07	10.16
		tf32	ours	0.0119	1.01	0.89	1.34	—	28.84
		bf16	ours	0.0083	0.95	0.84	1.69	2.23	43.63
16	384	fp32	ours	0.1060	0.91	0.94	1.50	2.01	16.51
		tf32	flex	0.0284	1.07	1.06	—	—	58.07
		bf16	flex	0.0163	1.28	1.26	2.63	4.13	102.00
16	512	fp32	ours	0.1674	0.77	0.82	1.24	1.67	33.40
		tf32	flex	0.0445	1.09	1.04	—	—	115.54
		bf16	flex	0.0242	1.11	0.99	2.35	3.62	218.40
32	192	fp32	ours	0.0549	0.70	0.75	1.05	1.48	7.71
		tf32	ours	0.0168	0.83	0.87	1.11	—	21.99
		bf16	flex	0.0103	0.98	0.91	1.56	2.14	36.60
32	384	fp32	ours	0.1884	0.82	0.85	1.13	1.66	9.96
		tf32	flex	0.0507	1.29	1.38	1.69	—	33.54
		bf16	flex	0.0250	1.38	1.36	2.11	3.03	67.97
32	512	fp32	ours	0.3174	0.67	0.71	0.92	1.28	17.83
		tf32	flex	0.0797	1.25	1.28	—	—	84.33
		bf16	flex	0.0393	1.20	1.22	1.81	2.61	147.88

kernel is faster than dense at one retained peer (0.56 to 0.68), ranges from 0.76 to 1.13 at two, and is slower at four everywhere (Appendix B.5); the share spent finding the set is 91 to 97% there.

4.5 Q4: Does the crossover depend on precision?

The single-peer advantage above is measured in fp32 with IEEE products, which keeps every kernel off the tensor cores so that it reproduces the reference summation order. That is the format in which dense attention is least advantaged. We repeated the comparison in TF32 and bf16, searching both kernels again in each format under the same admissibility filter, and added FlexAttention (Dong et al., 2025) twice: as a dense baseline, and as PyTorch's route to data-dependent sparsity, with the top-k retained set expressed as a block mask (Table 4).

Tensor cores help dense attention far more than they help the resident fused top-k kernel. The best dense implementation becomes 2.7 to 4.0 times faster in TF32 than in fp32, and 3.9 to 8.1 times faster in bf16. The resident kernel at one retained peer, faster than dense at every agent count in fp32 (0.67 to 0.91), is slower from 384 agents upward in both reduced formats (1.07 to 1.38) and at most marginally faster at 192 agents (0.83 to 1.01). One reduced-precision cell is at parity rather than slower under the ratio of medians (0.99, bf16, d=16, N=512). This separates the two readings that the fp32 data could not: at a fixed head width, raising the efficiency of dense attention removes the resident kernel's crossover, so it depends on how close dense attention runs to the machine rather than on the head width as such. TF32 also changes the function computed. Rounding the products reorders near-tied scores, so every top-k implementation — fused, composed or FlexAttention — retains a different peer from a float64 reference in up to 0.7% of query rows, rising with k; we exclude configurations above 0.1%, which removes most TF32 entries beyond one retained peer, and since cost rises with k those entries could only be slower.

FlexAttention is the fastest dense implementation in nine of the twelve reduced-precision configurations, which supports the choice of baseline. As a route to top-k sparsity it is 8 to 218 times slower than dense

Table 5: Selecting before scoring, at 4 retained peers: selector latency (ms) and end-to-end latency (selector plus fused attention) divided by the best dense implementation of the same interleaved sweep. The grid selector's recall is the fraction of exact nearest neighbours it retains. The Z-order path is shown on minima and as a ratio of medians, and through FlexAttention with the window expressed as a band mask. Rows below N=192 are dominated by per-kernel cost (Section 4.7).

	all-pairs kNN		**grid kNN**			**Z-order, fused**		**Z-order**
N	select	total	select	total	recall	min	median	**Flex**
48	0.048	8.12	0.047	8.09	1.0000	15.73	11.04	17.63
96	0.099	7.72	0.098	7.54	1.0000	8.62	7.69	14.97
192	0.317	10.08	0.506	16.09	1.0000	3.79	3.58	4.63
384	1.813	16.93	1.185	11.17	1.0000	1.26	1.34	2.25
512	5.692	37.48	1.669	9.85	0.9999	0.86	0.92	1.61

attention: the retained set must first be ranked from a full score matrix and converted into a block mask on every call, and at 128-wide blocks every block of the mask contains a retained pair, so the kernel then computes densely.

The streaming kernel in reduced precision. Repeating the study for the streaming kernel separates what reduced precision removes (Table 11 in Appendix B.3). At one retained peer the streaming kernel stays faster than dense in TF32 (0.62 to 0.82) and is at parity in bf16 (0.85 to 1.06), where the resident kernel was slower from 384 agents up. At two it stays faster in the TF32 configurations that pass the retained-set filter (0.78 to 1.01) and is slower in bf16 (1.10 to 1.66); at four it is 1.76 to 2.71 times slower in bf16, and no TF32 configuration passes. Reduced precision accelerates every matrix product, the score product included, but not the integer comparisons of selection, so the narrower the format the larger the share of the sparse kernel's time that selection takes. On the A100, H100 and H200 the one-peer ratio is 0.83 to 1.33 in TF32 and 1.03 to 1.52 in bf16: with tensor cores the selection no longer pays for itself (Appendix B.5).

4.6 Q5: Selecting before scoring

We test Proposition 2 with three selectors, each used by existing multi-agent mechanisms and each in a different class of Corollary 1. *All-pairs nearest neighbours* ranks the distances between two-dimensional positions, as a neighbourhood rule recomputed each step does (Jiang et al., 2020; Jiang & Lu, 2018); it is case (b), and its count ratio rises towards d and no further. *Grid nearest neighbours* buckets agents in a uniform grid and ranks only the candidates in the surrounding 3×3 cells; its cost is $\Theta(Nk)$ plus one sort of the agents, case (d) up to a sort that is under 5% of its count here, and it retains the exact nearest neighbours in at least 99.93% of cases for uniformly placed agents. *Z-order* orders agents along a Morton curve and keeps the k adjacent in that order, as ZETA does for low-dimensional key projections in sequence models (Zeng et al., 2025); its sort is $O(N \log N)$, case (c). Closed forms are in Appendix A.4. Attention then runs over the retained pairs only, composed from library operators or as the fused fixed-set kernel of Q3; the Z-order window, a band in curve order, also runs through FlexAttention. These selectors attend to different peers than top-k, so this comparison is of cost, not of quality; Table 2 is the comparison at a fixed set.

The attention stage. Once the retained set is known, the fused attention over it behaves as the accounting predicts: at two retained peers it costs 0.06 times dense attention at 512 agents, more than sixteen times faster, and its advantage grows with the agent count because the score term now shrinks with the aggregation (Figure 5 in Appendix B.9). The ratio of medians is 0.064 in that sweep but 0.18 in the second, more contended one, and under CUDA-graph replay the same kernel over the exact top-k set costs 0.07 (Table 2). The figure is a device time: the kernel runs for about ten microseconds, so an eager deployment whose host launch path is slower than that will not see it in full. Composed from library operators the same stage is slower than dense attention at every agent count and sparsity measured, and fusing it is worth a factor of 4

Table 6: Device time at the agent counts of current multi-agent methods, under CUDA-graph replay: the best dense implementation's median, and each implementation's median paired ratio against it (d=16, batch 256, fp32, three interleaved runs). The number under each name is the count of kernels it issues per call. The 192-agent row anchors the range of Sections 4.3–4.6.

N	best dense (µs)	fused, k=1 (1)	fused, k=2 (1)	fixed set, k=1 (2)	gathered, k=1 (8)	dense, unfused (6)	Z-order, k=1 (55)
12	4.0	0.99	1.06	1.48	6.32	3.66	20.87
24	5.9	0.94	1.02	1.02	4.97	2.74	14.81
48	8.1	0.95	1.02	0.78	5.78	2.74	12.24
96	14.6	0.85	0.99	0.48	6.84	3.23	7.64
192	35.0	0.77	1.32	0.29	9.20	4.02	3.33

to 17 (Appendix B.9). No score-then-select implementation comes close, because none can remove the score term.

End to end, the selector decides. The three selectors separate as Corollary 1 predicts, and by more than their operation counts suggest (Table 5). The all-pairs selector's cost grows quadratically, from 0.32 to 5.69 ms between 192 and 512 agents at four retained peers, and its path ends 37 times slower than dense attention; the cap binds far below its analytic value because the selector's cost is traffic rather than arithmetic. The grid selector removes the quadratic growth — its cost rises from 0.51 to 1.67 ms over the same range — but its constant, gathering and ranking about a hundred candidates per agent, keeps the path 10 times slower than dense at 512 agents, and at sixteen retained peers it is slower than the all-pairs selector at 384 and 512 agents: within the measured range the asymptotic class does not fix the ordering. The Z-order selector's cost is nearly flat, 0.10 to 0.14 ms, so its share falls as attention grows more expensive: the path costs 0.86 times dense at 512 agents and four retained peers, reproducing the first sweep's 0.85. Through FlexAttention the same window costs 1.6 times dense at 512 agents: faster than composing it from library operators, slower than the fused kernel, since at this size the band still touches several of FlexAttention's 128-wide blocks per row.

The end-to-end crossing is a best-case result. It holds on minima in both sweeps, and under the ratio of medians in the second sweep (0.92) but not in the first, where it is 2.10, because a multi-launch path has a heavier tail on a contended device than a single dense kernel (Appendix B.9). It appears only at the largest agent count we measure; at 48 agents the Z-order path is 16 times slower than dense. The section therefore identifies conditions — a selector cheap in absolute terms, not only asymptotically, and a fused attention stage behind it — rather than recommending this implementation. The crossing is also specific to the Blackwell card: on the A100, H100 and H200 the fused attention stage is still 0.16 to 0.24 times dense, but the whole Z-order path costs 1.12 to 1.45 (Appendix B.5).

4.7 Q6: What governs latency at the agent counts of current methods?

The multi-agent methods we cite are evaluated with two to about a hundred agents (Section 2), below the range in which the ratios above are read. There the per-call cost of a kernel is a large share of its time, so we re-timed every implementation at 12 to 96 agents under CUDA-graph replay, which removes host launch latency and leaves device time, and counted the kernels each one issues from the captured graph (Table 6).

At these sizes the number of kernels, not the operation count or the bytes, orders the implementations. Within each configuration of agent count and batch, the rank correlation between device time and kernel count has a median of 0.95, against 0.03 for executed operations; on the host, issuing a call costs about 9 µs per kernel on top of a fixed 61 µs, which an eager deployment pays in addition to the device time. Every composed path, sparse or dense, issues six to thirteen kernels and is 2.7 to 9.8 times slower than the best dense kernel at every agent count from 12 to 96, and the Z-order path, whose selector issues more than fifty, is 8 to 22 times slower, so the pre-score result of Section 4.6 does not extend below a few hundred agents in this implementation. Among single-kernel paths the resident fused score-then-select kernel at one retained

peer is at parity at 12 agents and 5 to 15% faster than dense from 24 to 96, and at two retained peers it is within 6% of parity throughout; the fixed-set kernel, which issues two kernels, is slower than dense at 12 agents and faster from 48. At realistic agent counts, then, the design question is how many kernels the communication step issues; whether it is sparse matters less.

5 Discussion and Limitations

5.1 What a designer can take from this

Each of the following is a conclusion from the measurements above, inside their envelope, rather than a general law. Charge the selector: its cost decides whether pre-scoring pays, and an asymptotic class does not fix the ordering at the agent counts measured. Separate scoring from aggregation: a rule that ranks scores cannot remove them, and in our fused kernels finding the retained set was most of the time. Check what the implementation executes: a masked implementation executes none of the counted saving. Check traffic before compacting, with $kd \leq N$ as a necessary screen. Compare against the strongest dense kernel available for the format, including reduced precision, which in bf16 confined the score-then-select advantage to a single retained peer. Count kernels at small agent counts, where the number launched orders latency. And keep attention-stage and end-to-end figures apart: a sixteenfold attention-stage speedup and a 2% spread across implementations in a full training iteration are both true of this system.

5.2 A checklist for benchmarking small kernels

Attention over tens to hundreds of agents runs for microseconds, on a device that is often shared, against library kernels tuned for other shapes. Each of those conditions produced at least one error in this study that yielded a plausible number, and Table 7 lists them with the practice that prevents each. Two of the eight deserve emphasis because nothing in the resulting numbers betrays them: a masked implementation benchmarked as sparse executes none of the counted saving, and a hand-written sparse kernel timed against a vendor's dense kernel measures authorship as well as sparsity. The first is caught by counting the operations the running program issues, the second by searching both kernels over one parameter space and reporting the fraction of the machine each reaches. Appendix C describes each entry and how it was found.

5.3 Limitations

Every cost result concerns centralised inference on a single device; the paper makes no claim about bandwidth-constrained distributed deployment, where reducing transmitted messages is itself the objective and top-k selection may well be the right design. The analysis is single-round and single-head; the operation-count ratio is invariant to both, but H heads of width d/H change the per-head width, along which our timing result changes sign.

The full measured envelope is one GPU architecture (NVIDIA Blackwell, sm_120), one x86_64 CPU, PyTorch 2.11 with CUDA 12.8 and Triton 3.6, head widths 16 to 512, $N \leq 512$ (1024 for the streaming kernel), batch 32 to 1024, and fp32, TF32 and bf16. The headline measurements — the composed and fused crossovers, the fixed-set split, the streaming kernel in all three formats, the pre-score path, small N and batch to 4096 — were repeated on an A100, an H100 and an H200 (Appendix B.5). The ordering holds on all four. The resident kernel's one-peer crossover is the same on all four, and the streaming and pre-score crossovers are narrower on the other three than on the Blackwell card, so those in the main text are best cases across the devices we measured. We did not measure CPUs or non-NVIDIA accelerators. Both cards were shared with other tenants throughout, and clocks could not be locked; every claim except the tiling search's efficiency figures is therefore a ratio between implementations timed in the same interleaved sweep, and below 192 agents only the CUDA-graph device times of Section 4.7 are used. Timing a single call with device events measures host launch latency whenever the host is slower than the kernel; our kernels run for 5 to 140 µs, and during the fixed-set sweep the host launch path was slow enough that the eager figure for the fixed-set kernel was 1.7 to 3.4 times its graph-replay figure. The score-then-select ratios agreed under both protocols, but the smallest device times we report are not what an eager deployment on a slower host would see.

Table 7: A checklist for benchmarking small attention kernels: eight ways a benchmark measures something other than the algorithm, each of which produced a spurious result in this study, and the practice that prevents it. The first five are caught by the shape of the result; the last three are not, because the numbers they produce are self-consistent. Appendix C describes each.

Kind	**Cause**	**Symptom**	**Corrective practice**
Instrumentation	Host transfer inside the timed region	Dense latency nearly flat in N	Gate diagnostics behind a flag that is off while timing
	Synchronisation without a device argument	Every kernel reports the event-resolution floor	Pin the device and pass it to every synchronisation
	Host allocator returning large temporaries	Up to 8.8× inflation and a non-monotone curve	Pin the heap; count minor faults per forward
Interference	Timing implementations in sequential blocks	Swings above 100% between identical configurations	Interleave repetition by repetition; report paired ratios
	Changing which implementations share a sweep	Two sweeps disagree by 1.4× at twelve agents	Fix the implementation set across comparable sweeps
Wrong algorithm	Benchmarking a masked implementation as sparse	The advertised saving is never executed	Assert executed FLOPs differ and trace the products issued
	Comparing only against unfused dense	A 5.4× more favourable comparison at N=512	Report both baselines; check which backend serves the shape
Wrong baseline	A hand-written sparse kernel against a vendor's dense kernel	An apparent speedup larger than the analytic ceiling permits	Write the dense baseline in the same framework and tiling

Our kernels are not optimal. The resident-row kernels admit only three tilings above 192 agents against 28 to 106 for the dense kernel, so their efficiency figures are lower bounds and the negative results at wide heads are weakly supported. The streaming kernel removes that constraint and moved the boundary from one retained peer to between two and four; it was searched at head widths 16 and 32 only, and its merge is a sequence of comparisons that a better selection network might shorten, so the boundary may move further. It cannot move past the score product, about half of dense attention. Its retained count is a compile-time constant and N is uniform within a launch, so a captured CUDA graph serves one (B, N, k); per-row k or ragged agent counts would need a length argument per row, which we have not written. We did not measure how the retained-set changes of reduced precision affect training: in the formats and configurations we report, bf16 changed no retained set and TF32 configurations that changed more than 0.1% of rows are excluded. The pre-score selectors attend to a different peer set than top-k does, so their comparison is of cost; the one quality check, Z-order in the learning study (Appendix D), is descriptive at six agents and five seeds; their selectors are composed from library operators, so the selector costs we report are upper bounds; and their envelope is one head width, one batch and fp32, with uniformly placed agents. Finally, the learning study (Appendix D) has five seeds per variant; it shows that the communicating architecture improves return and cannot rank sparsification variants.

6 Conclusion

We asked when an operation-count saving from sparse attention in multi-agent communication becomes a saving in time, and answered by charging every peer-selection rule for the cost of deciding. A rule that ranks scores must form all of them, so at most half of the forward-pass multiply–accumulates are removable; composed from library operators it is slower than dense attention everywhere we measure, and fused it pays only at one or two retained peers, because finding the retained set takes most of its time. Selecting before scoring is where the saving can become real, and the selector then decides the outcome. At the agent counts multi-agent methods use, the number of kernels a step launches decides latency before either. A speedup

claimed for sparse communication therefore has to be measured, on the kernel that runs and against the strongest dense baseline for the format.

Broader Impact Statement

This work is a measurement and correction study of a computational-efficiency claim. It introduces no new capability and no new data. Its intended effect is to reduce wasted compute in the design of multi-agent communication architectures by discouraging optimisation against an operation count that does not predict runtime.

Reproducibility Statement

The kernels, measurement harness, analysis scripts and every measurement artifact are provided as anonymised supplementary material. Two scripts regenerate every table and figure from the artifacts; a third re-derives the numbers quoted in the main text from a manifest that names each one's source artifact and tolerance, and fails on any disagreement. The environment, commands and measurement parameters are in Appendix H.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL https://arxiv.org/abs/2108.13264.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document Transformer. *arXiv preprint*, 2020. URL https://arxiv.org/abs/2004.05150.

Zhaodong Chen, Zheng Qu, Yuying Quan, Liu Liu, Yufei Ding, and Yuan Xie. Dynamic N:M fine-grained structured sparse attention mechanism. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pp. 369–379, 2023. doi: 10.1145/3572848.3577500. URL https://doi.org/10.1145/3572848.3577500.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with Performers. *International Conference on Learning Representations (ICLR)*, 2021. URL https://arxiv.org/abs/2009.14794.

Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-Awareness. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.

Abhishek Das, Théophile Gervet, Joshua Romoff, Dhruv Batra, Devi Parikh, Michael Rabbat, and Joelle Pineau. TarMAC: Targeted multi-agent communication. *International Conference on Machine Learning (ICML)*, 2019. URL https://arxiv.org/abs/1810.11187.

DeepSeek-AI. DeepSeek-V3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025. URL https://arxiv.org/abs/2512.02556.

Mostafa Dehghani, Anurag Arnab, Lucas Beyer, Ashish Vaswani, and Yi Tay. The efficiency misnomer. In *International Conference on Learning Representations (ICLR)*, 2022.

Juechu Dong, Boyuan Feng, Driss Guessous, Yanbo Liang, and Horace He. Flex attention: A programming model for generating optimized attention kernels. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025.

William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL https://jmlr.org/papers/v23/21-0998.html.

Ankit Gupta, Guy Dar, Shaya Goodman, David Ciprut, and Jonathan Berant. Memory-efficient Transformers via top-k attention. In *Proceedings of the Second Workshop on Simple and Efficient Natural Language Processing (SustaiNLP)*, pp. 39–52. Association for Computational Linguistics, November 2021. doi: 10.18653/v1/2021.sustainlp-1.5. URL https://aclanthology.org/2021.sustainlp-1.5/.

Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, pp. 1–12. ACM, 2015. doi: 10.1145/2807591.2807644.

Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefler. Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 3, 2021.

Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with Gumbel-Softmax. *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1611.01144.

Jiechuan Jiang and Zongqing Lu. Learning attentional communication for multi-agent cooperation. *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. URL https://arxiv.org/abs/1805.07733.

Jiechuan Jiang, Chen Dun, Tiejun Huang, and Zongqing Lu. Graph convolutional reinforcement learning. *International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/1810.09202.

Daewoo Kim, Sangwoo Moon, David Hostallero, Wan Ju Kang, Taeyoung Lee, Kyunghwan Son, and Yung Yi. Learning to schedule communication in multi-agent reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2019. URL https://openreview.net/forum?id=SJxu5iR9KQ.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient Transformer. *International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/2001.04451.

Xunhao Lai, Weiqi Xu, Yufeng Yang, Qiaorui Chen, Yang Xu, Lunbin Zeng, Xiaolong Li, Haohai Sun, Haichao Zhu, Vito Zhang, Jinkai Hu, Jiayao Li, Rui Gao, Zekun Li, Songquan Zhu, Jingkai Zhou, and Pengyu Zhao. MiniMax sparse attention. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2606.13392. URL https://arxiv.org/abs/2606.13392.

Yen-Cheng Liu, Junjiao Tian, Chih-Yao Ma, Nathan Glaser, Chia-Wen Kuo, and Zsolt Kira. Who2com: Collaborative perception via learnable handshake communication. In *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6876–6883, 2020a. doi: 10.1109/ICRA40945.2020.9197364.

Yong Liu, Weixun Wang, Yujing Hu, Jianye Hao, Xingguo Chen, and Yang Gao. Multi-agent game abstraction via graph attention neural network. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 7211–7218, 2020b. doi: 10.1609/aaai.v34i05.6211.

André F. T. Martins and Ramón Fernandez Astudillo. From softmax to sparsemax: A sparse model of attention and multi-label classification. *International Conference on Machine Learning (ICML)*, 2016. URL https://arxiv.org/abs/1602.02068.

Yaru Niu, Rohan R. Paleja, and Matthew C. Gombolay. Multi-agent graph-attention communication and teaming. In *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, pp. 964–973, 2021. URL https://www.ifaamas.org/Proceedings/aamas2021/pdfs/p964.pdf.

Daniel Ohayon, Itay Lamprecht, Itay Hubara, Israel Cohen, Daniel Soudry, and Noam Elata. Block sparse flash attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. URL https://arxiv.org/abs/2512.07011. To appear; arXiv:2512.07011.

Georgios Papoudakis, Filippos Christianos, Lukas Schäfer, and Stefano V. Albrecht. Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks. *Advances in Neural Information Processing Systems Datasets and Benchmarks Track (NeurIPS)*, 2021. URL https://arxiv.org/abs/2006.07869.

Ben Peters, Vlad Niculae, and André F. T. Martins. Sparse sequence-to-sequence models. *Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1504–1519, 2019. doi: 10.18653/v1/P19-1146. URL https://aclanthology.org/P19-1146/.

Markus N. Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint*, 2021. URL https://arxiv.org/abs/2112.05682.

Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with Routing Transformers. *Transactions of the Association for Computational Linguistics (TACL)*, 9: 53–68, 2021. doi: 10.1162/tacl_a_00353. URL https://aclanthology.org/2021.tacl-1.4/.

Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1701.06538.

Amanpreet Singh, Tushar Jain, and Sainbayar Sukhbaatar. Learning when to communicate at scale in multiagent cooperative and competitive tasks. *International Conference on Learning Representations (ICLR)*, 2019. URL https://arxiv.org/abs/1812.09755.

Adam Stooke, Joshua Achiam, and Pieter Abbeel. Responsive safety in reinforcement learning by PID Lagrangian methods. *International Conference on Machine Learning (ICML)*, pp. 9133–9143, 2020. URL https://proceedings.mlr.press/v119/stooke20a.html.

Sainbayar Sukhbaatar, Arthur Szlam, and Rob Fergus. Learning multiagent communication with backpropagation. *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. URL https://arxiv.org/abs/1605.07736.

Chuangchuang Sun, Macheng Shen, and Jonathan P. How. Scaling up multiagent reinforcement learning for robotic systems: Learn an adaptive sparse communication graph. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 11755–11762, 2020. doi: 10.1109/IROS45743.2020.9341303. URL https://arxiv.org/abs/2003.01040.

J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, Niall L. Williams, Yashas Lokesh, and Praveen Ravi. PettingZoo: Gym for multi-agent reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL https://arxiv.org/abs/2009.14471.

Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, pp. 10–19. ACM, 2019. doi: 10.1145/3315508.3329973.

Muning Wen, Jakub Grudzien Kuba, Runji Lin, Weinan Zhang, Ying Wen, Jun Wang, and Yaodong Yang. Multi-agent reinforcement learning is a sequence modeling problem. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2205.14953.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785. URL https://doi.org/10.1145/1498765.1498785.

Yaodong Yang, Rui Luo, Minne Li, Ming Zhou, Weinan Zhang, and Jun Wang. Mean field multi-agent reinforcement learning. *International Conference on Machine Learning (ICML)*, 2018. URL https://arxiv.org/abs/1802.05438.

Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative, multi-agent games. *Advances in Neural Information Processing Systems Datasets and Benchmarks Track (NeurIPS)*, 2022. URL https://arxiv.org/abs/2103.01955.

Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Yuxing Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 23078–23097. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1126. URL https://aclanthology.org/2025.acl-long.1126/.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big Bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL https://arxiv.org/abs/2007.14062.

Qiuhao Zeng, Jerry Huang, Peng Lu, Gezheng Xu, Boxing Chen, Charles Ling, and Boyu Wang. ZETA: Leveraging Z-order curves for efficient top-k attention. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2501.14577.

Guangxiang Zhao, Junyang Lin, Zhiyuan Zhang, Xuancheng Ren, Qi Su, and Xu Sun. Explicit sparse transformer: Concentrated attention through explicit selection. *arXiv preprint arXiv:1912.11637*, 2019.

A Proofs and assumptions

A.1 Proposition 1

Proof. From (1), $C_{\text{top-}k} = N^2d + Nkd > N^2d = \frac{1}{2}C_{\text{dense}}$ for every $k \geq 1$, since $Nkd > 0$, so the ratio $2N/(N+k)$ is below 2. It is decreasing in k, hence largest at $k = 1$, and approaches 2 from below as $k/N \to 0$. □

We use N^2d rather than $N(N-1)d$ because it is the count the implementations execute: self-masking writes $-\infty$ into the diagonal of a full $N \times N$ score matrix and does not skip work. Counting the diagonal exactly gives $2(N-1)/\big((N-1)+k\big)$, which is strictly smaller for every $1 \leq k \leq N-1$, so the proposition is a valid upper bound under either convention.

Seven assumptions support the statement. The second is essential; the sixth and seventh fix the constant. **(A1) Single round**: with R rounds both terms scale by R and the ratio is unchanged. **(A2) Full-score selection**: the retained set is determined only after the complete candidate score vector has been computed, so all N scores must be formed. **(A3) Per head**: with H heads of width d/H both terms scale identically. **(A4) Projections excluded**: including the shared qkv and output projections moves the ratio strictly closer to 1, so the bound remains valid. **(A5) Selection cost excluded**: counting the top-k operation itself can only decrease the ratio, and Appendix B.6 shows the term is large in practice. **(A6) Equal key and value widths**: with key width d_k and value width d_v the two products cost N^2d_k and N^2d_v, and the ceiling is $1 + d_v/d_k$, which is 2 in the equal-width attention used throughout and exceeds two when values are wider than keys. **(A7) Element-wise work excluded**: the scale, softmax and exponentials are not counted. Charging α operations per softmaxed element and σ per ranked score gives a ceiling $(2d+\alpha)/(d+\sigma)$, which exceeds two when $\alpha > 2\sigma$; at d=16 with five operations per softmax element and one comparison per score it is 2.09.

A2 concerns *when* the retained set is fixed, not whether the rule depends on the data. Routing Transformer's online k-means routing (Roy et al., 2021) and Reformer's locality-sensitive hash (Kitaev et al., 2020) are content-dependent in every ordinary sense and still evaluate scores only for candidate pairs; a hierarchical rule that ranks compressed blocks and then scores only the selected ones (Yuan et al., 2025) is likewise outside A2. A2 also does not require the scores to be held at once: a streaming selector that keeps a running candidate set computes all N scores without storing them, and remains inside the class.

A.2 The training-time analogue

Exact selection is piecewise constant in the scores, so gradients reach only the retained entries. Of the four products in dense attention's backward pass — the gradients with respect to values, weights, queries and keys, each N^2d — a top-k backward needs each only over the Nk retained pairs, given the indices recorded in the forward pass. Counting both passes for such a backward, without recomputation,

$$\frac{C^{\text{f+b}}_{\text{dense}}}{C^{\text{f+b}}_{\text{top-}k}} = \frac{6N^2d}{N^2d + 5Nkd} = \frac{6N}{N+5k} < 6, \tag{4}$$

so in training at most five sixths of attention's multiply–accumulates are removable, against half in inference. The composed gathered path we time does not reach this bound: autograd scatters the retained-score gradient into a dense $N \times N$ tensor and forms the query and key gradients as dense products, so its executed count is $3N^2d+3Nkd$ and its ratio is $2N/(N+k)$, as in the forward pass. A dense implementation that recomputes the scores in its backward pass, as FlashAttention does, adds N^2d to the dense count; against a top-k backward that stores its indices the supremum becomes 7, a difference in what the dense baseline recomputes rather than in what sparsity removes. The measurements of Appendix B.7 show the gathered path's backward is cheap relative to its forward because selection has almost no adjoint, not because its backward arithmetic is sparse.

A.3 Proposition 2 and Corollary 1

Proof of Proposition 2. Both statements read off the displayed ratio. For (i), $2Nkd + C_{\text{sel}} = 2Nkd\,(1 + o(1))$, so $S = (N/k)\,(1 + o(1))^{-1}$, whose quotient with N/k tends to 1; substituting $C_{\text{sel}} = 0$ gives $2N^2d/2Nkd = N/k$ exactly, and any $C_{\text{sel}} > 0$ strictly enlarges the denominator. For (ii), if $k/N \to 0$ then $2Nkd = o(N^2d)$, so the whole denominator is $o(N^2d)$ and the ratio diverges. □

For a score-then-select rule the displayed form charges the Nk retained scores inside $2Nkd$ although the rule has already formed them while selecting, so its selector cost in this form is at least the scores of the discarded pairs, $C_{\text{sel}} \geq N^2d - Nkd$. That is a lower bound and not an estimate: the ranking itself is uncounted, and counting it enlarges C_{sel} and shrinks the ratio further. Substituting the bound gives $2N^2d/(N^2d + Nkd)$, which is Proposition 1.

Proof of Corollary 1. Each case substitutes its C_{sel} into the denominator of S. Case (a) is Proposition 1. In (b) the $\gamma N^2 p$ term dominates $2Nkd$ at fixed k and d, giving $S \to 2N^2d/(\gamma N^2 p)$. In (c) the $\Theta(N \log N)$ term likewise dominates $2Nkd$, giving $S = \Theta(N^2d/(N \log N))$, and $S/(N/k) = \Theta(kd/\log N) \to 0$; the statement needs $kd = o(\log N)$; with $d = \Theta(\log N)$ the penalty is a constant factor, and it disappears only when kd grows faster than $\log N$. In (d), $S = 2N^2d/((2 + c)Nkd) = 2N/((2 + c)k)$ exactly, which is $\Theta(N/k)$ but not asymptotic to it; with $C_{\text{sel}} = o(Nkd)$ it is Proposition 2(i). □

The selector-cost term is material, not decorative. A fixed topology or a precomputed spatial prior really is free at inference and lands in case (d), which is why Longformer and BigBird obtain asymptotic savings where score-then-select top-k cannot (Beltagy et al., 2020; Zaheer et al., 2020): their patterns are fixed before any score is formed, so nothing is spent deciding. A hash must be evaluated and its buckets assembled (Kitaev et al., 2020), and a routing rule must maintain and query its clusters (Roy et al., 2021), so their C_{sel} has to be counted, not assumed away. Between the two sits a family of multi-agent methods whose selector is itself pairwise — a learned hard-attention or scheduling network evaluated over all N^2 pairs before communication attention runs (Liu et al., 2020b; Niu et al., 2021) — and for these C_{sel} is of the same order as the score matrix they avoid, so they inherit a ceiling of the same kind instead of escaping it.

A.4 Closed forms for the measured selectors

The three selectors of Section 4.6, charged under the unit convention of Corollary 1, have closed-form ratios. The all-pairs nearest-neighbour selector charges three operations for each two-dimensional distance and one comparison per candidate, $4N^2$ operations or $2N^2$ MAC-equivalents per batch element, giving $Nd/(kd + N)$; the ranking is charged one comparison per candidate, a lower bound for any selection algorithm, so the formula overstates that selector's ratio. The Z-order selector quantises, interleaves ten bit-pairs and sorts, $N(15 + \lfloor \log_2 N \rfloor + 1)$ MAC-equivalents, giving $4Nd/(4kd + 32 + 2\lfloor \log_2 N \rfloor)$. At 512 agents and four retained peers the first has reached 14.2 against its ceiling of $d = 16$, while the second is at 107 — unbounded in principle, and already only 84% of the $N/k = 128$ that the naive count promises. The grid selector charges one sort to bucket agents and, per agent, a distance and a comparison for each of $9M$ candidates in the surrounding cells, where M is the per-cell capacity; for spatially uniform positions $M = \Theta(k)$, so the candidate term is $\Theta(Nk)$ and the bucketing sort adds $\Theta(N \log N)$. With a linear-time bucketing the selector is in case (d) with $c = \Theta(1/d)$ (about 3.5 at d=16, k=4); as implemented, with a comparison sort, it is formally case (c), though the sort is under 5% of the charged cost at every agent count we measure.

A.5 Machine balance and arithmetic intensity

We compute arithmetic intensity, floating-point operations per byte of global memory traffic, analytically per stage, counting each tensor read and written once, and compare it against a machine balance measured on the same device: a large matrix multiply gives the achievable arithmetic rate, a large copy gives achievable bandwidth, and their ratio is the ridge point above which a kernel is compute-bound. Counting each tensor once ignores cache residency, so at small N, where the working set fits the GPU's L2, the analytic byte counts overstate traffic; the per-stage accounting of Appendix B.6 flags such stages so the reader need not infer

them. The ridge point itself depends on whether the probe's buffer fits the last-level cache, so we measure two of them per device. On the GPU the ridge is 23 FLOP/byte cache-resident and 53 from memory; on the host it is 8 and 28. Both composed paths sit below every one of those figures at every agent count at d=16, on both devices, so the memory-bound conclusion does not depend on which ridge is the relevant one. The per-stage table of Appendix B.6 quotes a third figure, 39, because its probe runs in the same process as the stage timings it scales and measures the arithmetic rate at 57.8 rather than 76.7 TFLOP/s on the same card; every stage sits below that ridge as well. Appendix F.5 gives the probe.

B Measurement details

B.1 Protocol

We time the communication path with the training loop removed. Timing uses on-device CUDA events with an explicit synchronisation on the pinned device, and any diagnostic that transfers a scalar to the host is disabled, because such a transfer synchronises the device inside the timed region. Implementations are timed interleaved repetition by repetition rather than in sequential blocks, so that interference from other tenants lands on all of them at once and their ratios stay paired; the sequential design produced swings above 100% between identical configurations on this shared host. One interleaved group holds every implementation at one agent count, including every retained-peer count of every sparse implementation and the dense implementations they are divided by, so a ratio never spans two groups. The released artifacts record the dense rows with $k = N - 1$, which is a label and not a separate measurement. The primary statistic is the minimum over samples, on the modelling assumption that contention adds time rather than removing it, and the whole timed block is repeated so that the spread of per-run medians is reported and not hidden (Appendix E.2). A minimum estimates an uncontended envelope rather than a service time, so we checked what the choice buys: recomputing every comparison at 192 agents and above from the medians of the same samples changes which implementation is faster at one of 24 configurations, 384 agents with a single retained peer, where the ratio moves from 0.94 to 1.14 and the group's recorded inter-run spread is 64%. Neither of the two sweeps that measure the same configuration reproduces the reversal. The pre-score path of Section 4.6 is the one place where the estimator does change a conclusion, and we report it there. GPU clocks could not be locked; the account lacks the privilege. Operation counts are analytic, from a model that follows the convention `torch.utils.flop_counter` uses and that we checked against it on the composed implementations; the hand-written kernels are outside what that counter observes. Every count in this paper is therefore a model, and where a count and a measurement disagree the measurement governs — which is, after all, the paper's subject.

Two decisions about what is compared deserve stating, because both changed our conclusions. First, we time the attention core — from queries, keys and values to the message tensor — rather than the enclosing module, since the shared input and output projections are identical across implementations and dilute the quantity in question. Second, the dense baseline is the better of PyTorch's fused scaled-dot-product path and a dense attention kernel we wrote ourselves in Triton, with the same tiling policy and in the same session as the sparse kernels. At the head widths multi-agent communication uses, the library path is tuned for a different regime, and comparing a hand-written sparse kernel against it credits sparsity with the difference between two authors (Section C).

Every implementation is checked against the reference at every configuration whose latency any artifact reports — 158 of them, 136 with $k < N - 1$ so that selection is exercised. This is not a formality: exact top-k is discontinuous in the scores, so two implementations that accumulate the score product in different orders can rank a near-tied pair differently and then aggregate over different peers. All agree to `fp32` rounding except on four query rows in 1.25 million, where two scores tie exactly at the k-th position and the threshold-selecting kernel keeps one extra peer (Appendix F).

Unless stated otherwise: PyTorch 2.11 with CUDA 12.8 and Triton 3.6, `fp32`, head width d=16, batch 256, and one NVIDIA Blackwell GPU shared with other tenants.

Table 8: The sparsity at which the best sparse implementation stops being faster than the best dense one, and how far it is from parity at a quarter of the peers. Both ratio columns are a minimum over the composed path and the two fused kernels that rank in registers, so the widest-k column is sometimes the composed path; the sorted kernel of Table 10 is faster than all three from sixteen retained peers upward and is reported there. Restricted to the agent counts at which per-kernel cost does not dominate. The replayed column is the fused kernel at one retained peer under CUDA-graph replay, pooled over three further sessions of three interleaved blocks each, as ratio of minima and ratio of medians.

	sparsity that pays		**best ratio**		**replayed,** k=1	**at the widest** k	
N	k	of peers	ratio	at k	min / median	ratio	at k
192	≤ 1	0.5%	0.76	1	0.76 / 0.77	27.9	128
384	≤ 1	0.3%	0.94	1	0.92 / 0.92	18.8	128
512	≤ 1	0.2%	0.81	1	0.77 / 0.77	15.4	128

Table 9: Both kernels searched over one parameter space — query tile, key tile or head chunk, warp count and pipeline depth — under one shared-memory admissibility filter, with the better of the search and the vendor library taken as the dense baseline. The *cfgs* columns are how many configurations each search admitted; the sparse kernel must hold an N-wide score tile, so above 192 agents only three fit. Efficiency is executed arithmetic over elapsed time, against the matrix-multiply rate the device attains in the same process. These are the search session's own timings, taken configuration by configuration rather than interleaved; the two search sessions measured peaks of 58 and 71 TFLOP/s, so the efficiency columns are comparable within a row but not across sessions. Every ratio elsewhere in the paper re-times the selected configurations in an interleaved sweep.

		best dense				**best sparse**					
d	N	from	cfgs	ms	% peak	k	cfgs	ms	% peak	ratio	k that pays
16	192	triton	106	0.0347	30	1	12	0.0249	21	0.72	≤ 1
16	384	triton	106	0.1082	38	1	3	0.0941	22	0.87	≤ 1
16	512	triton	106	0.1681	44	1	3	0.1293	29	0.77	≤ 1
32	192	triton	94	0.0555	38	1	18	0.0428	24	0.77	≤ 1
32	384	triton	94	0.1923	43	1	3	0.1618	26	0.84	≤ 1
32	512	triton	94	0.3179	47	1	3	0.2231	33	0.70	≤ 2
64	192	sdpa	64	0.0862	48	1	21	0.0962	22	1.12	none
64	384	sdpa	64	0.2720	61	1	3	0.4321	19	1.59	none
64	512	sdpa	64	0.4361	68	1	3	0.5634	26	1.29	none
128	192	sdpa	28	0.1665	50	2	18	0.1724	24	1.04	none
128	384	sdpa	28	0.4688	71	1	3	0.8582	19	1.83	none
128	512	sdpa	28	0.7656	77	1	3	1.1784	25	1.54	none

B.2 The fused crossover and three selection strategies

That statement has to be qualified in turn, and the qualification is the more useful result. Matched authorship is a weaker condition than matched efficiency: two kernels written by the same person in the same session can still differ by more than the margin being measured, and only efficiency against the machine identifies the algorithm. We therefore searched the same parameter space for both kernels under the same admissibility filter, took the better of that search and the vendor library as the dense baseline, and report what fraction of the device's own matrix-multiply rate each kernel reaches beside its latency (Table 9).

The search is not equal. All three of our sparse kernels keep the whole score row of a query resident — two rank it in registers and one sorts it in place — so their resident tile grows with the agent count, and above 192 agents the shared-memory filter admits only three configurations against the dense kernel's twenty-eight

Table 10: Three ways of reaching the same retained set, at the two agent counts that carry the crossover. Repeated argmax keeps the aggregation sparse and pays linearly in the number retained; bisecting on the score's bit pattern is flat in it and pays the dense aggregation; sorting score–index pairs once is flat and sparse at the same time, and is not competitive with either at the agent counts where a sort no longer fits alongside a useful block of queries.

		k **argmax rounds**		**bisection**		**one sort**	
N	k	ms	÷dense	ms	÷dense	ms	÷dense
192	1	0.026	0.8	2.392	69.4	0.164	4.7
	2	0.045	1.3	2.435	70.7	0.166	4.8
	4	0.069	2.0	2.435	70.7	0.168	4.9
	8	0.116	3.4	2.435	70.7	0.173	5.0
	16	0.206	6.0	2.441	70.8	0.185	5.4
	32	0.385	11.2	2.440	70.8	0.223	6.5
512	1	0.139	0.8	2.970	17.2	2.433	14.1
	2	0.217	1.3	2.716	15.8	2.450	14.2
	4	0.337	2.0	2.764	16.0	2.493	14.5
	8	0.557	3.2	2.912	16.9	2.490	14.5
	16	1.002	5.8	2.911	16.9	2.316	13.4
	32	1.884	10.9	2.920	16.9	2.651	15.4

to a hundred and six. That is a property of these kernels and not of exact top-k: the streaming kernel of Section 4.4 computes all N scores without holding them and admits the dense kernel's tilings, so the search counts here describe the resident kernels only. We report the counts in the table rather than describing the two searches as equivalent, and the consequence runs in two directions. The sparse kernel's efficiency figure is a lower bound on what a sparse kernel of this shape could reach, so the negative entries at head width 64 and above are weakly supported and we do not build on them. The positive entry is conservative for the same reason: an under-searched sparse kernel that is nonetheless faster is faster.

With that said, two things follow.

The crossover survives tuning, and it is not produced by a sparse kernel that happens to be well written. Across head widths 16 and 32 the sparse kernel is the *less* efficient of the two at every agent count — 21 to 33% of the device's rate against 30 to 47% for dense — and it is faster only because it performs about half the arithmetic.

The crossover is an existence result inside a stated envelope, and the envelope is narrow: one GPU architecture, `fp32`, head widths 16 and 32, agent counts from 192 upward where the sweep reproduces, a single retained peer with one configuration at two, and a dense baseline that reaches under half of the device's arithmetic rate at those head widths. At four retained peers the resident kernel is already 1.3 to 2.3 times slower, and at a quarter of peers it is more than an order of magnitude slower; the streaming kernel of Section 4.4 moves the boundary to between two and four retained peers. The k=1 case is also degenerate as an attention mechanism, because a softmax over one element is identically one and the mechanism reduces to copying the highest-scoring peer's value. What the crossover establishes is that the penalty measured earlier is not irreducible; it is not a recommendation to sparsify.

Why it is confined to narrow heads could not be decided from the fp32 data alone. Dense efficiency and head width move together there — 30% at d=16 rising to 77% at d=128, as the score product becomes a better-shaped matrix multiply — so "the crossover disappears once dense attention is near the machine" and "the crossover disappears above d=32" fit it equally well. The precision study of Section 4.5 separates them: at a fixed head width, a dense kernel that uses the tensor cores removes the crossover from 384 agents upward, which supports the first reading.

Selection is what a successor design has to face, and we tried three ways of doing it. Ranking N candidates to keep k by repeated argmax costs k sequential row-wide reductions, and a reduction of that shape runs at

Table 11: The streaming kernel of Table 3 in reduced precision, searched separately in each format, against the best dense implementation of the same format and interleaved sweep (ratios of minima under CUDA-graph replay); the resident kernel at one retained peer is re-timed in the same sweep. A dash marks a configuration in which TF32 rounding changed the retained set in more than 0.1% of query rows; bf16 changed none.

			streaming, ÷ best dense			**resident**
format	d	N	k=1	k=2	k=4	k=1
TF32	16	192	0.82	1.01	—	0.90
	16	384	0.78	—	—	1.08
	16	512	0.73	—	—	1.10
	32	192	0.70	0.88	—	0.83
	32	384	0.62	0.78	—	1.32
	32	512	0.62	—	—	1.28
bf16	16	192	1.06	1.34	2.09	1.06
	16	384	0.99	1.58	2.61	1.33
	16	512	0.96	1.66	2.71	1.10
	32	192	0.96	1.14	1.76	0.93
	32	384	0.92	1.15	2.02	1.39
	32	512	0.85	1.10	1.92	1.22

a small fraction of the arithmetic units' throughput because each round depends on the one before it and the hardware has nothing to overlap it with. That is why the argmax kernel's cost grows almost linearly in k while its arithmetic barely does. Bisecting on the score's bit pattern removes the dependence on k, and the only way it can is by expressing the retained set as a zero weight instead of by compaction — so it performs the dense aggregation arithmetic whatever k is.

Whether both can be had at once is the obvious question, and they can (Table 10). Packing each score against its own peer index in one machine word and sorting the row once yields the retained peers already compacted: the aggregation then reads exactly k value rows, while selection costs a single bitonic sort, whose depth grows with the agent count alone and not at all with k. That is visible in what the kernel costs. At 192 agents it runs in 0.164 ms at one retained peer and 0.185 ms at sixteen, a rise of thirteen per cent, against an eightfold rise for the argmax kernel over the same range; what remains of the growth is the aggregation, which is linear in k by construction, and not the selection.

It is also five times the cost of dense attention across that whole range, and it overtakes the argmax kernel only at sixteen retained peers, where both already cost more than five times dense. At 384 and 512 agents it is worse still, thirteen to sixteen times dense at every sparsity below thirty-two retained peers, because a row-wide sort needs the whole score row resident and the block of queries that can share a program shrinks as the row grows. Having both properties does not help. The sort that buys independence from k costs more than the argmax rounds it replaces throughout the range in which sparsity would otherwise pay.

The second obstacle is one dense attention avoids and none of the resident-row kernels does. Dense attention streams keys and values in blocks under an online softmax, keeping a small tile resident whatever N is. Each of those kernels instead holds the N-wide score row — in registers to rank it, or in place to sort it — so fewer queries can be in flight per streaming multiprocessor as N grows, and the tiling the search chooses for it is narrower at every size. What is fundamental to exact score-then-select is that all N scores must be *computed* before the retained set is known; that they must all be *resident* is not, and a streaming exact top-k that keeps a running candidate set relaxes it. The streaming kernel of Section 4.4 does exactly that, and is faster than every resident kernel at every configuration both cover (Table 3).

Table 12: The headline quantities on four GPUs. Each device's tilings are searched on that device before any sweep times them, and each quantity is computed by the function that computes it for the main text (`scripts/summarise_portability.py`); every entry is a ratio against the best dense implementation of the same interleaved sweep, so a value below one means the sparse path is faster. Streaming ranges are over $d \in \{16, 32\}$ and the agent counts of Table 3, under CUDA-graph replay; the other rows are defined as in the main-text tables they summarise. The Blackwell card was shared with other tenants; the other three were allocated exclusively, one GPU per job, on a shared node. The A100 runs PyTorch 2.11 and Triton 3.6 built against CUDA 12.9, the others the same versions built against CUDA 12.8.

	RTX PRO 6000 (sm_120)	A100 (sm_80)	H100 SXM (sm_90)	H200 (sm_90)
Composed gathered, min over $N \geq 192$ and k (fwd)	9.01	6.57	6.75	7.65
Composed gathered, min (backward)	1.53	2.60	2.03	2.10
Resident fused, k=1	0.67–0.91	0.69–0.87	0.69–0.89	0.69–0.89
Streaming fp32, k=1	0.46–0.63	0.56–0.64	0.56–0.68	0.56–0.68
Streaming fp32, k=2	0.56–0.84	0.77–0.96	0.76–1.13	0.76–1.13
Streaming fp32, k=4	0.78–1.15	1.03–1.50	1.05–1.45	1.05–1.47
Streaming fp32, k=8	0.92–1.76	1.28–2.96	1.25–2.04	1.25–2.06
Streaming TF32, k=1	0.62–0.82	0.83–1.05	0.83–1.03	0.88–1.33
Streaming bf16, k=1	0.85–1.06	1.03–1.30	1.10–1.32	1.11–1.52
Share of fused time finding the set ($N \geq 384$)	90–95%	91–95%	91–96%	93–97%

Table 13: The streaming kernel against the best dense implementation as the batch varies, d=16, agent counts 192 to 512, both kernels searched at every batch (ratios of minima under CUDA-graph replay). Heads are independent, so H heads over a batch B are one launch over a batch BH: this is also the multi-head packing sweep.

	A100 (sm_80)			H100 SXM (sm_90)			H200 (sm_90)		
B	k=1	k=2	k=4	k=1	k=2	k=4	k=1	k=2	k=4
32	0.60–1.08	0.86–1.86	1.41–1.83	0.71–1.17	1.00–1.00	1.45–1.90	0.71–0.76	0.95–1.24	1.34–1.94
256	0.58–0.64	0.87–0.96	1.29–1.50	0.61–0.65	0.86–0.94	1.35–1.45	0.61–0.64	0.85–0.94	1.35–1.47
1024	0.57–0.62	0.87–0.91	1.26–1.40	0.60–0.64	0.86–0.87	1.36–1.40	0.60–0.64	0.86–0.87	1.37–1.40
4096	0.57–0.60	0.83–0.88	1.22–1.38	0.59–0.62	0.84–0.85	1.32–1.39	0.59–0.62	0.84–0.85	1.32–1.39

B.3 The streaming kernel in reduced precision

B.4 Sensitivity to head width and batch

Across head widths to 512, and at the agent counts from 192 upward that carry claims, the crossover exists only at the two narrowest: the best sparse implementation is faster than the best dense one at head widths 16 and 32 at every agent count, and at no agent count for head width 64 or above, where it is 1.3 to 2.3 times slower (Appendix E). The launch-bound 96-agent column has one exception, 0.88 at head width 64, which we do not read as a result. The ratio does not fall monotonically with head width; it reverses.

That is what the byte criterion predicts. Byte-neutrality requires $kd \leq N$, so widening the head at fixed agent count tightens the bound on the retained set proportionally, and at head width 128 the criterion permits only $k \leq 4$ at 512 agents (exactly $k \leq 3$) — by which point the selection cost of Section 4.4 has already overtaken the saving. Widening the batch over a thirty-twofold range, by contrast, moves the ratio by up to a third — from 0.54 to 0.72 at 512 agents — and never across parity, which is the expected behaviour of a quantity that is a ratio of per-element costs.

B.5 Other GPU generations

We repeated the measurements of Section 4 on an A100-SXM4 (40 GB, sm_80), an H100 SXM (80 GB HBM3, sm_90) and an H200 (141 GB HBM3e, sm_90), with every tiling searched again on each device (`scripts/run_portability_campaign.sh`). Table 12 collects the quantities the main text rests on. The ordering holds on all 3 devices: the composed path is at least 6.6 times slower than dense attention; from

Table 14: Per-stage accounting for both paths at N=512. The last column is the fraction of whichever machine limit binds the stage, and it separates a stage that is badly implemented from one doing the only thing its access pattern permits. The byte counts are analytic and count each tensor once, so a figure above 100% marks a stage whose working set is partly served from cache. That happens for the softmax stages here and, at smaller agent counts, for the gather; it bounds the model's error at around +50 percentage points, which is larger than several of the gaps in the column and is why we do not read small differences off it. Peak measured on the same device: 57.8 TFLOP/s and 1497 GB/s, a ridge point of 39 FLOP/byte.

stage	GFLOP	GB	FLOP/byte	ms	TFLOP/s	GB/s	% of limit
score product	2.15	0.29	7.53	0.549	3.91	520	35
softmax over N	0.34	0.54	0.62	0.336	1.00	1598	107
dense aggregation	2.15	0.29	7.53	0.235	9.13	1213	81
top-k selection	0.00	0.47	—	3.271	—	143	10
softmax over k	0.08	0.13	0.62	0.059	1.42	2268	151
gather of V	0.00	1.21	—	0.954	—	1265	84
sparse aggregation	0.53	1.14	0.47	0.816	0.65	1398	93

384 agents upward 91 to 97% of the fused kernel's time is finding the set; and the streaming kernel is faster than dense at one retained peer in fp32 (0.56 to 0.68). Beyond that the streaming kernel's advantage shrinks. At two retained peers the streaming kernel ranges from 0.76 to 1.13, so it is no longer faster at every configuration; at four it is slower everywhere (1.03 to 1.50). At one retained peer it costs 0.83 to 1.33 times dense in TF32 and 1.03 to 1.52 in bf16, so in bf16 it does not break even on any of the three. The Z-order pre-score path at 512 agents and four retained peers is not faster on any of them: its fused attention stage costs 0.16 to 0.24 times dense, but with the selector the path costs 1.12 to 1.45. The resident kernel's one-peer ratio is the same on all four devices; for the streaming kernel and the pre-score path the Blackwell card is the most favourable of the four, and the crossovers the main text reports for them are best cases.

Batch and heads. Table 13 varies the batch from 32 to 4096 on the three devices. From 256 upward the threshold does not move: at one retained peer the streaming kernel costs 0.57 to 0.65 times dense and at two 0.83 to 0.96. At a batch of 32, where a launch has only a few programs per multiprocessor, the one-peer ratio widens to 0.60 to 1.17. Packing more heads into a launch therefore cannot widen the window; it can only bring a small batch back to the large-batch ratio.

Hardware counters. On the H100 and the H200 we recorded Nsight Compute counters for every kernel each implementation launches (`scripts/ncu_counters.py`; the A100 nodes do not expose the profiling counters to users). At 512 agents and four retained peers the composed path moves 4.4 GB through DRAM in 27 kernels, against 28 MB for the tuned dense kernel; 17 of those kernels are `torch.topk`'s multi-block radix select, which makes four passes over the score matrix and takes 81 to 82% of the path's time. The composed baseline is thus already a production radix top-k, and its cost is the byte criterion's: compaction adds traffic. The fused and streaming kernels move about as many bytes as dense attention and reach only 0.9 to 1.6% of peak DRAM throughput, against 52 to 60% of peak SM throughput. Selection inside a fused kernel is bound on the multiprocessors, not by bandwidth, which is consistent with the H200, with 1.4 times the H100's measured DRAM bandwidth, giving the same ratios.

B.6 Where the time goes

Low arithmetic intensity in a batched matrix–vector product is the natural explanation for the sparse contraction's slowness, and it is not the right one. Timed in isolation and charged with its own arithmetic and traffic, both aggregations sit near the bandwidth the device attains on a large copy — 93% for the sparse contraction and 81% for the dense multiply it replaces (Table 14). The gap between them is smaller than the byte model's own error, so the model cannot order the two; what it does establish is that neither sits on a worse hardware path. The sparse one is slower because it moves about four times the bytes, since the

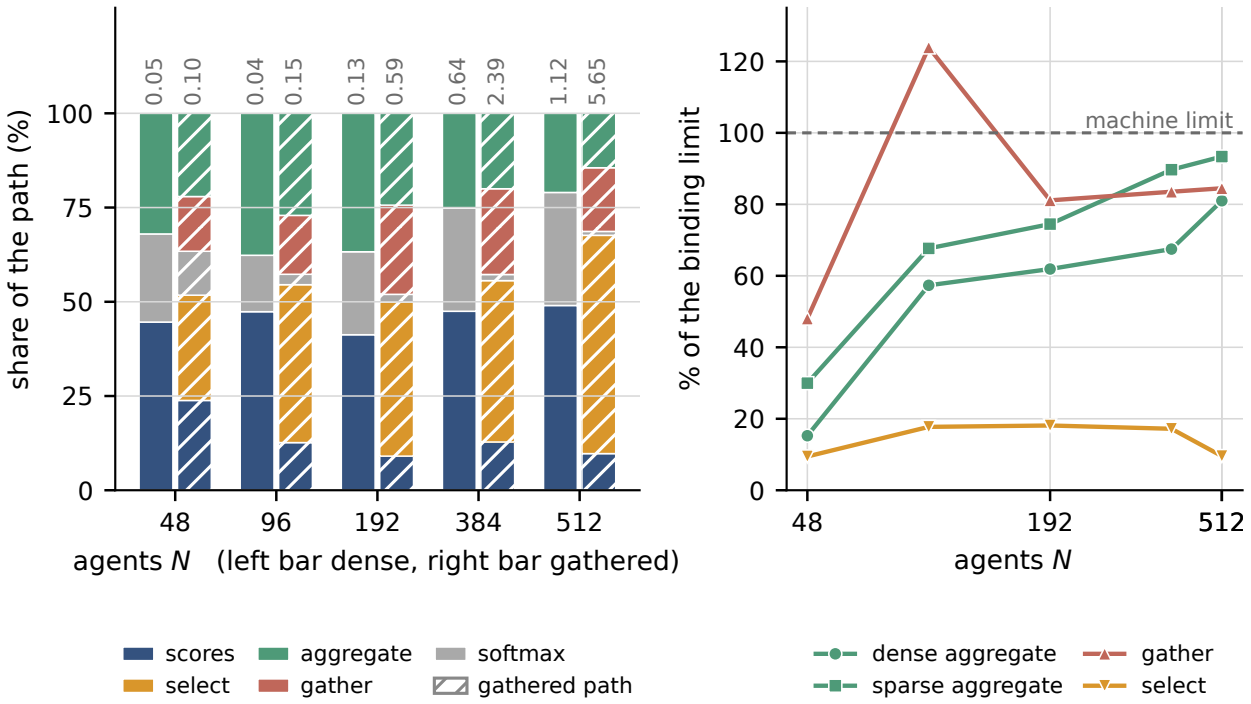


Figure 4: **Left:** how each path divides its time. Selection grows to dominate the gathered path as the agent count rises. **Right:** how close each stage runs to the machine limit that binds it.

Table 15: Forward, backward and their sum, for the differentiable implementations. The sparse path's backward is cheap relative to its forward, because selection has almost no adjoint, so the gap against fused dense narrows without closing. Restricted to the agent counts that reproduce: below 192 the forward and the forward-plus-backward passes are timed in separate interleaved blocks, and the contention between blocks exceeds the backward's own cost, which at two configurations produced a difference a backward pass cannot have.

		dense, unfused			dense, fused			top-k masked			top-k gathered			
N	k	f	b	f+b	f	b	f+b	f	b	f+b	f	b	f+b	gathered f+b ÷ fused f+b
192	32	0.136	0.166	0.302	0.072	0.219	0.291	0.454	0.460	0.914	0.424	0.201	0.625	2.1
384	64	1.082	1.604	2.686	0.220	0.728	0.948	3.065	3.365	6.430	2.405	1.593	3.998	4.2
512	128	1.899	2.648	4.547	0.349	1.203	1.552	6.985	5.748	12.734	6.435	3.142	9.577	6.2

compacted operand carries kd elements per query where the score row carries N. The corrected account is simpler, checkable before the code is written, and it is the $kd \leq N$ criterion of Section 3.4.

Selection is a different case, and it is the one stage where the raw decomposition is misleading in the opposite direction. It is the largest stage of the composed sparse path, and it runs at under a tenth of the bandwidth the device attains: it must read the score matrix and write values and indices, and moving those bytes and nothing else would take a small fraction of the time it does. That headroom is real, and the fused kernels realise part of it. What no kernel removes is that the score matrix must be formed and ranked at all, and the fused measurements of Section 4.4 show what remains once the headroom is taken.

B.7 Backward pass and training throughput

Every latency reported so far is a forward pass, which for a training claim is half the question. Timing forward and backward under the same protocol leaves the ordering intact and roughly a third of the margin

Table 16: Where one MAPPO iteration spends its time. Differences between communication implementations are at or below the iteration-to-iteration spread in the rightmost column.

		share of the iteration (%)							
N	communication	environment	acting	update fwd	update bwd	optimiser	s/iter	÷ fused	spread
6	none	49	29	11	6	4	2.11	1.002	4.6
6	dense, unfused	48	30	12	6	4	2.09	0.992	7.3
6	dense, library fused	48	30	12	6	4	2.10	1.000	4.0
6	top-k, masked	48	30	12	6	4	2.09	0.996	5.4
6	top-k, gathered	49	29	12	6	4	2.13	1.011	5.3
12	none	76	14	5	2	2	4.35	0.998	2.3
12	dense, unfused	75	14	6	4	2	4.39	1.007	3.8
12	dense, library fused	75	14	6	3	2	4.36	1.000	3.3
12	top-k, masked	75	14	6	3	2	4.39	1.008	5.6
12	top-k, gathered	75	14	6	4	2	4.41	1.011	2.3
24	none	92	5	2	1	1	12.84	1.002	4.2
24	dense, unfused	91	5	2	1	1	12.98	1.013	3.3
24	dense, library fused	91	5	2	1	1	12.81	1.000	1.6
24	top-k, masked	91	5	2	1	1	12.80	0.999	0.8
24	top-k, gathered	91	5	2	1	1	12.76	0.996	1.7

(Table 15). The gathered path's backward costs about half its own forward, because the expensive stage — selection — has almost no adjoint: the retained indices are recorded in the forward pass, and the backward is a scatter of gradients into the retained positions followed by two dense products. Dense attention's backward costs several times its forward. At 512 agents the gathered path is 18.4 times the fused dense path on the forward alone and 6.2 times on forward and backward together, and the same narrowing appears at every agent count in the table. The sparse path's disadvantage in training is therefore substantially smaller than the forward-only measurements suggest, and remains a disadvantage. The fused kernels are excluded from this comparison: they are hand-written forward passes, and timing one against an autograd graph would compare two different things.

End to end, none of this is visible. Timing a complete training iteration by phase, with the communication implementation swapped and everything else fixed, puts the environment and per-step action selection at 78% of the iteration at six agents and 96% at twenty-four, and leaves all five implementations within 2% of one another at every agent count, against an iteration-to-iteration spread of 0.8 to 7.3% (Table 16). The ordering among the five is not stable: the communication-free variant, which performs no attention at all, is fourth of five at six agents and at twenty-four, and the gathered sparse implementation is the fastest of the five at twenty-four. This does not resolve the implementations as equal; it fails to resolve them at all, which is the useful description as well as the honest one. It bounds the practical stakes of the whole measurement campaign in the regime the training study occupies, and it is why the computational argument is made on the module in isolation.

B.8 Robustness

The ordering is unchanged under `torch.compile` and CUDA-graph capture, at `bf16`, and on an x86_64 CPU whose machine balance differs from the GPU's by more than an order of magnitude. The CPU comparison required controlling an effect unrelated to attention: with the default allocator the multi-MiB temporaries each implementation frees between calls are returned to the kernel and re-faulted on the next call, which at N=192 adds roughly 37,000 minor page faults per forward and inflates latency by up to 8.8×. The tax is proportional to the bytes of large temporaries each implementation churns, so it does not cancel in a ratio, and in an un-pinned run it inverted the ordering at one configuration. With the heap pinned the inversion disappears. Appendix F.4 gives the procedure and both runs.

Because clocks are unlocked and the host is shared, we re-ran the identical sweep with the device otherwise idle and again under a concurrent load. With the device idle the ratios reproduce to within a few per cent; under load absolute latency moves by more than 100% while the paired ratios move by roughly half that,

and every ratio moves against the sparse path. Pairing removes the typical drift rather than the tail, so the quantity that matters is the margin: no condition we measured brings a composed sparse implementation below parity, and the fused crossover reported in Section 4.4 is measured on the quietest device state we could obtain. Appendix E.3 gives the figures.

B.9 Selecting before scoring in detail

The paragraphs below describe the first pre-score sweep, which timed the all-pairs and Z-order selectors (Table 17, Figure 5); the second sweep of Section 4.6 adds the grid selector and FlexAttention and reproduces its Z-order and all-pairs rows.

Table 17: Selecting before scoring, at 2 retained peers, every ratio against the better of the vendor library and our own dense kernel at the tiling the search of Section 4.4 found. Composing the attention stage from library operators costs an order of magnitude more than fusing it, and the difference decides the conclusion: composed, the attention is slower than dense attention at every configuration measured; fused, it is up to sixteen times faster.

		ops	**measured (ms)**			**best**	**÷ best dense**		
				attend			attend		total
N	selector	saved	select	composed	fused	dense	composed	fused	fused
48	nearest neighbours	10	0.049	0.030	0.005	0.008	3.96	0.60	6.91
48	Z-order window	18	0.102	0.031	0.005	0.008	3.99	0.60	13.79
96	nearest neighbours	12	0.100	0.045	0.006	0.014	3.22	0.44	7.58
96	Z-order window	36	0.113	0.045	0.006	0.014	3.23	0.44	8.44
192	nearest neighbours	14	0.324	0.075	0.007	0.034	2.20	0.20	9.51
192	Z-order window	71	0.121	0.077	0.007	0.034	2.24	0.19	3.71
384	nearest neighbours	15	1.757	0.137	0.010	0.108	1.27	0.09	16.42
384	Z-order window	140	0.124	0.138	0.009	0.108	1.28	0.08	1.23
512	nearest neighbours	15	5.449	0.177	0.010	0.172	1.03	0.06	32.41
512	Z-order window	184	0.129	0.177	0.010	0.172	1.03	0.06	0.83

Fused, the escape delivers, and how much depends on how many peers are kept. At two retained peers the attention stage costs 0.60, 0.44, 0.19, 0.08 and 0.06 times dense attention from 48 to 512 agents; at four, 0.79, 0.46, 0.24, 0.12 and 0.09. The best case over the grid is 0.060, sixteen times faster than dense, at 512 agents and two retained peers. The advantage narrows as k grows and is gone at the smallest agent count by sixteen retained peers, where the stage costs 1.06 times dense behind the Z-order selector and 1.39 behind the nearest-neighbour one, whose retained rows are scattered rather than adjacent. That shape is what the accounting predicts once the score term shrinks with the aggregation, and no score-then-select implementation in this paper comes close to it at any sparsity, because none of them can remove the score term.

Composed from library operators, the same stage is slower than dense attention at every one of the forty configurations, by 1.03 to 5.58 times. Fusing it is worth between 4.0 and 17.3 times, and the factor grows with the agent count because the gathered tensor the composition writes and re-reads grows with it. We report both columns because the composed one is what we measured before writing the kernel, and on its own it supports the opposite conclusion.

End to end the selector decides the outcome, and the two selectors behave oppositely. The Z-order selector's cost is nearly flat in the agent count — 0.102 to 0.142 ms across the range — so its share of the path falls as attention grows more expensive: at four retained peers the whole path costs 13.9, 8.7, 3.8, 1.28 and 0.85 times dense attention from 48 to 512 agents. The nearest-neighbour selector moves the other way. It materialises an $N \times N$ distance matrix and ranks it, which is Corollary 1(b), so beyond 96 agents its cost grows quadratically and its path rises to 32.3 times dense.

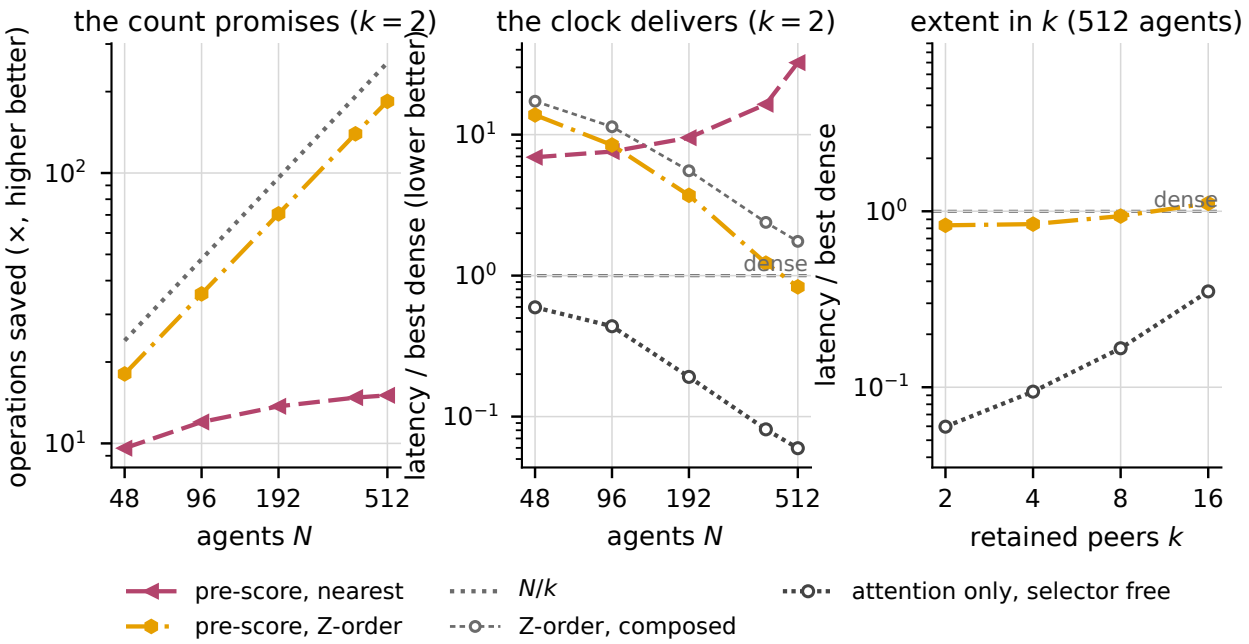


Figure 5: First pre-score sweep. **Left:** the operation-count saving the accounting of Section 3.3 promises. The dotted line is N/k, the figure a free selector would reach; the sub-quadratic selector stays below it by a factor that widens with N, and the all-pairs selector is capped by a constant (Corollary 1). **Middle:** what the same configurations cost. Fused, the attention over pre-chosen peers is more than an order of magnitude faster than dense attention at the largest agent count, and the total is then governed by the selector, which is where the two selectors diverge. **Right:** how far the end-to-end crossing extends. It holds at two and four retained peers, is level with dense at eight and gone by sixteen.

The end-to-end crossing at 512 agents needs a qualification the attention stage does not. It is a best-case result and it does not survive a change of estimator. Every latency in this paper is a minimum over interleaved samples, an estimate of what the configuration costs when the shared device is not being disturbed. Under that estimator the Z-order path costs 0.83 times dense at two retained peers and 0.94 at eight, and the two runs of the sweep agree to within 2.3%. Under the ratio of medians of the same samples it costs 2.06 times dense instead. The reason is visible in the dispersion: the pre-score path is several library kernels behind one selector, and on a contended device the tail of a multi-launch path is much heavier than the tail of a single dense kernel, so the median charges it for contention that the minimum removes from both sides. We therefore report the crossing as an uncontended-envelope result and make no claim about end-to-end cost under load. The attention stage carries no such caveat: it is 16.8 times faster than dense on minima and 15.7 on medians.

Three things follow, and together they are the paper's constructive result.

Escaping the operation-count ceiling is necessary and not sufficient. Both selectors escape it, by between 7 and 107 times in operations at four retained peers, and one of them is an order of magnitude slower than dense attention while the other is faster. What separates them is whether the selector reaches its decision through a structure of the size it was introduced to avoid. Corollary 1 says as much before any measurement: the nearest-neighbour selector is capped at a constant by its own all-pairs structure. What the measurement adds is that the cap binds far below its analytic value, because the selector's cost is traffic rather than arithmetic and the count charges only the arithmetic.

A cheap selector is not sufficient either. With the attention stage composed from library operators the Z-order path is slower than dense at every agent count we measure, and only the fused kernel brings the total to parity at all. Both conditions are needed, and an operation count expresses neither.

And the regime matters, in a way that limits what this result recommends. The crossing appears at 512 agents, the largest count we measure, and we do not extrapolate past it. Most of the multi-agent communication methods we cite are evaluated with two to roughly thirty agents, and DGN and ATOC with about a hundred (Section 2); at those counts our own measurements put the pre-score path behind dense attention, by 13.9 times at 48 agents and 1.28 at 384. So the negative result of Section 4 is the one that bears on current practice, and this section is a demonstration of a mechanism rather than a recommendation to adopt this implementation. What it supplies to a designer is a question with a checkable answer in place of an appeal to an operation count: whether the selector avoids a structure of the size it was introduced to remove, and whether the sparse attention it leaves behind is executed in a kernel written for it.

C Sources of measurement error

This appendix expands the checklist of Table 7. Each entry produced a spurious result during this study before it was caught, and in each case the first symptom was a plausible number rather than an obvious error. The causes group into four kinds, and the grouping is useful because the kinds differ in how they are caught.

Instrumentation. A diagnostic that synchronises the device inside the timed region measures the synchronisation. Our attention module computed an entropy statistic through a call that transfers a scalar to the host, and with it enabled dense latency was nearly flat in N while its work grew by two orders of magnitude — a result we briefly read as evidence that the quadratic cost does not manifest. Synchronising without naming the device is the complementary error: work submitted to a non-default device is never waited on, and every kernel at every size then reports the event-resolution floor of about $0.4\,\mu$s. On the host, the default allocator returns the multi-MiB score matrices each implementation frees between calls, so the next call re-faults them; at N=192 that added roughly 37,000 minor page faults per forward and inflated latency by up to 8.8×. Errors of this kind leave traces — an impossible latency, a curve that is flat where it should be quadratic, a non-monotone cost — and are caught by looking at the shape of the result rather than its value.

Interference from a shared device. Timing implementations in sequential blocks makes each block hostage to whatever else was running during it. On our shared host the recorded inter-run spread reaches 374% at one configuration of the released sweeps, and the sessions that produced our tiling searches differ from each other by up to 81% on identical shapes. Interleaving the implementations repetition by repetition lands any interference burst on all of them at once, so the ratios between them survive even when the absolute numbers drift. The residual sensitivity is to which implementations share an interleaved sweep: at small N, where a fixed per-call cost is a large share of the measurement, the launch count of the whole group enters every measurement in it, and two of our own sweeps that differ only in which implementations they time disagree by 1.4× at twelve agents. Sweeps that will be compared must therefore hold their implementation set fixed and say so.

Measuring a different algorithm from the one claimed. The two most dangerous errors are of this kind, because the resulting numbers are internally consistent, reproducible, and stable under every check the others would fail. They answer a different question from the one being asked. Applying top-k as a mask leaves the dense aggregation matmul in place, so the advertised saving is never executed; a FLOP model based on $1 - k/(N-1)$ nonetheless credits that code with a 75% saving at N=512, k=127, and with more than 90% wherever $k/(N-1) < 0.1$. Comparing against unfused dense alone is the same error in the baseline: at N=512 the unfused and fused dense paths differ by 5.4×, so which one is meant sometimes changes the answer. The defence is to state the operation count each implementation executes alongside its latency, and to check which backend actually serves each shape and dtype, both of which make a mismatch between the claimed and the executed algorithm visible immediately.

Attributing implementation quality to the algorithm. The fourth kind appeared only when we wrote our own kernels. Timing a hand-written sparse kernel against a vendor library's dense kernel measures sparsity and authorship at once. At the head widths used in multi-agent communication, PyTorch's fused

Table 18: The two positive controls: final return of the communicating variant minus the non-communicating one, five seeds each. Both were inspected before the analysis plan for the remaining comparisons was registered and are reported descriptively.

	$A - B$	95% interval	p
communicating − non-communicating, reference task	+4.26	[+3.15, +5.23]	0.0079
communicating − non-communicating, partially observable	+11.59	[+7.13, +16.44]	0.0079

attention path runs at roughly half the rate of a straightforward Triton kernel, because it is tuned for the long sequences and wide heads of language models. Against it our fused sparse kernel appears up to 2.8× faster, which is more than sparsity can supply at any k and was how we noticed. Matched authorship is the obvious remedy and it is not sufficient either: two kernels by the same author can differ by more than the margin under test. The condition that identifies the algorithm is matched *efficiency*, so we search one parameter space for both under one admissibility filter — which admits very unequal numbers of configurations, and we report those counts too — and give the fraction of the device's own arithmetic rate each kernel reaches (Section 4.4). That is also what reveals when neither implementation is near the machine, which is a different warning and one worth having: a comparison between two kernels at a third of peak resolves the algorithms only if the margin exceeds the headroom, and ours does not everywhere.

One belief of our own also needed correcting. We assumed that standard counters are blind to fused attention kernels, which would make a FLOPs-only comparison unable to see the baseline that matters. In PyTorch 2.11 that is not so: `torch.utils.flop_counter` reports the fused path correctly, and it was our harness that added an analytic term on top of the counter's output for that implementation, publishing a figure 1.83× too large. The narrower point generalises better than the one we started with: a counter has to be validated against the kernels it is expected to observe, because only a counter attached to the running program can contradict a FLOP model.

D Learning study

Sparsification would still be worth having if it cost nothing in return, and testing that requires a channel that is actually used: on a task where it is not needed, "sparsification is free" and "the channel was never used" make identical predictions. The role of this study is therefore narrow — to establish that the architecture being sparsified does something — and no claim elsewhere in this paper rests on it.

We train with MAPPO (Yu et al., 2022) on PettingZoo/MPE (Terry et al., 2021), five seeds per variant and two million environment steps, following the protocols of Papoudakis et al. (2021) and Agarwal et al. (2021). Two positive controls test whether the channel carries something: a reference task in which each agent observes only its partner's goal, fixed at two agents where selection is degenerate, and a partially observable variant of a standard cooperative task, introduced here, in which a peer's relative position is withheld beyond a fixed radius (Appendix G.4).

Both controls separate completely, each attaining the floor of an exact permutation test at five seeds (Table 18), and the measured attention entropy on the partially observable control is 0.35 nats against a ceiling of 1.61, so the learned weighting is peaked rather than uniform. Sparsification is therefore applied to an architecture whose communication machinery improves return under the evaluated setup. It does not follow that the attention channel alone accounts for the difference: the communicating variants carry 7–19% more trainable parameters, and the channel-specific claim would need a parameter-matched sham variant, which we do not have. Comparisons between sparsification variants were pre-registered with a power calculation that showed in advance they could resolve only complete separation at five seeds; all are Holm-adjusted nulls, and they are reported with the pre-registration in Appendix G.

A selector chosen before scoring, in training. The Z-order selector of Section 4.6 is compared there on cost alone, so we added it to this study after the fact, as a descriptive and unregistered variant on the partially observable task at six agents and two retained peers, identical to the attention top-k variant except

for the selector. It reads each agent's own position from its observation, pooled across agents through the channel, and no other simulator state. Over five seeds its final return is -197.8 ± 5.8, against -193.4 ± 5.5 for attention top-k, -197.5 ± 7.7 for uniformly random peers and -204.4 ± 3.3 without communication. The difference from no communication is $+6.7$, with a 95% interval of $[+1.5, +11.8]$; from random peers -0.3 $[-7.7, +7.4]$, and from attention top-k -4.3 $[-10.5, +2.1]$. The cheap selector therefore keeps the benefit of communicating, but on this evidence no more than random peers do: at six agents 49.5% of the peers a Morton window selects are among the two true nearest neighbours, against 40% for a random choice, so curve locality is weak at this size. Whether a selector cheap enough to pay in time is also informative enough to matter is a question for larger populations, where this one has more locality to exploit.

E Additional benchmark results

E.1 The full parameter grid

Table 19 gives the best-over-implementations sparse latency divided by the best-over-implementations dense latency, across head width and across batch, at each agent count and at the most favourable sparsity available. Head width is swept to 512 and batch from 32 to 1024. The largest batch we attempted, 4096 at 512 agents, could not be measured: the score matrix alone runs to several gigabytes and the device is shared, so that configuration is absent.

Table 19: Best sparse divided by best dense, over head width (upper block) and batch (lower block). Values below one indicate sparsity winning. The 96-agent column is included for completeness and carries no claim, for the reason given in Appendix E.3. Each column comes from the sweep that varies that axis, so a figure here can differ from the same nominal quantity in Section 4.4 by the few per cent that separates two sessions. The searched dense tiling exists only at batch 256, so in the batch block our dense kernel runs at its default tiling except at batch 256; in the head-width block it is missing at four configurations with $d \geq 64$, where the search's winner was the library kernel, which is in the baseline.

head width d	N=96	N=192	N=512
16	0.85	0.76	0.82
32	0.51	0.79	0.70
64	0.88	1.29	2.23
128	1.41	2.19	1.84
256	2.07	2.04	2.05
512	1.54	1.71	2.31

batch B	N=96	N=192	N=512
32	0.54	0.65	0.54
1024	0.57	0.75	0.72

E.2 Dispersion behind the headline numbers

The primary statistic is the minimum over interleaved samples, on the modelling assumption that contention can only add time. That statistic is trustworthy only if the underlying sample was not wildly dispersed, so the harness records the median, the interquartile range and the spread of per-run medians for every configuration, and the released artifacts carry all of them. Two properties of this host limit what any of it can mean. Both GPUs carried other tenants' allocations and near-continuous utilisation throughout the campaign — tens of gibibytes, varying between sessions, which we observed but did not record per run — and clocks could not be locked. We therefore report the spread of per-run medians beside every configuration. Whether a bootstrap interval for a ratio of minima is meaningful depends on the contention process, which is not stationary on this host, so we do not report one for the minimum.

E.3 Drift control

The headline sweep was repeated in full on the same device, hours later and under different load from other tenants. Table 20 gives how far each agent count moved between the two runs, in absolute latency and in the ratio against the best dense implementation, which is the quantity every claim is stated in.

Table 20: Movement between two runs of the identical sweep, hours apart under different load. Median movement is at or below one per cent everywhere except 512 agents, where it is six to seven per cent; no single configuration moves by more than 12.6%, and no ordering the paper reports changes between the runs.

		latency $\lvert\Delta\rvert$ (%)		**ratio** $\lvert\Delta\rvert$ (%)	
N	configurations	median	max	median	max
96	44	0.1	4.2	0.3	2.5
192	44	0.2	6.1	0.2	6.0
384	44	0.7	11.3	0.5	11.6
512	44	6.1	12.0	7.2	12.6

Across 44 configurations at each agent count the median movement in latency is 0.1% at 96 agents, 0.2% at 192, 0.6% at 384 and 6.1% at 512, and in the ratio 0.3, 0.2, 0.5 and 7.2%; the worst single configuration moves 12.6%. Which implementation is faster changes between the runs at exactly one configuration, 96 agents with two retained peers, where the ratio is 1.002 in one run and 0.998 in the other — a tie either way.

Two things follow for how the sweeps are read. The measurement reproduces well enough that every ordering in this paper survives repetition, which is the property the claims depend on. And the largest agent count is the least stable one, not the most, so a margin of a few per cent there — which is what the end-to-end crossing of Section 4.6 is — is at the edge of what this device can resolve, and we say so where we report it. We restrict the crossover of Section 4.4 to 192 agents and above for a different reason: fixed per-call cost. A one-program Triton kernel that writes a single float, timed the way every measurement here is timed, costs 1.8 µs at its minimum and 2.3 µs at its median (`scripts/launch_overhead.py`). At 96 agents the fastest dense configuration is 13.8 µs and the fastest fused sparse one 11.7 µs, so that fixed cost is an eighth of each, and the composed sparse path issues several kernels rather than one. At 512 agents the same cost is one per cent of the dense measurement. A ratio between implementations that issue different numbers of kernels is therefore partly a ratio of launch counts at the small agent counts, and stops being one by 192.

The pre-score sweep of Section 4.6 is repeated in the same way, and for a stronger reason: the crossing it reports is a margin of about ten per cent rather than a factor, and a margin of that size has to be shown to survive a second session before it can be reported at all. It does. The median movement in the ratio between the two runs is 0.7% and the worst is 8.5%; at the agent count that carries the crossing the median is 0.8% and the worst 3.6%, and the four cells the crossing itself is read off agree to within 1.5%. Unlike the core sweep, this one reproduces at every agent count it measures, which is why the section quotes it from 48 agents upward.

The repeat is on the same physical card, so it establishes stability across runs and hours, not independence from one device, and we were not able to obtain a second card free of other tenants. The tiling search of Table 9 was run twice, once on each card of the host. The two searches select the same configuration at 63 of the 72 shapes they share, and at head width 16, where most of what this paper reports lives, at 16 of the 18 shapes; the two disagreements are both sparse-kernel tilings. Their absolute times differ by as much as 81% with the contention each session carried. That is why the search contributes a configuration and not a latency: every sweep re-times that configuration in its own interleaved group, and the only place a latency from the search's own session appears is Table 9, whose columns are the search's own record and are labelled as such. All measurements reported here were taken on one of the two cards of this host, and no cross-device calibration is claimed.

F Implementation and measurement validation

F.1 Functional equivalence

Every implementation is checked against the reference at every configuration whose latency any released artifact reports: 158 combinations of agent count, head width and retained-peer count, 136 of them with $k < N - 1$ so that selection is actually exercised, over 1.25 million query rows in total. The check matters more than it looks. Exact top-k is discontinuous in the scores, so two implementations that accumulate the score product in different orders can rank a near-tied pair differently and aggregate over different peers, producing a large output difference from a rounding-level score difference. An earlier version of our per-query fused kernel computed the score row as a reduction instead of through the matrix-multiply units and disagreed with the reference at a handful of configurations for exactly that reason; the kernels reported here use the same product as the reference.

With that fixed, every implementation agrees with the reference to `fp32` rounding — maximum deviation 3.2×10^{-6}, and zero for the two implementations that share the reference's operator sequence exactly — with one attributable exception, which is an ambiguity in the function rather than an error in an implementation. Exact top-k is only well defined when the k-th largest score is unique. The kernel that selects by threshold retains every peer at or above that score, so a tie there gives it $k{+}1$ peers; the kernel that sorts score–index pairs breaks the same tie by peer index, which is not the order a rank-based selection uses. Four query rows in 1.25 million have such a tie, at three of the 158 configurations, and those rows are the only place any implementation differs from the reference by more than rounding. Two further rows appear in the head-width sweep, at 192 agents and head width 128, where the same two kernels differ from the reference by 1.7 and 1.1 on one query row of 6144 at each of two retained-peer counts; the k-th score on those rows is tied to the bit, so the divergence is the ambiguity and not an error. The check is run by `scripts/verify_all_configurations.py` and by the sweep harness itself, both of which count how many query rows diverge as well as by how much, and admit a tie as the explanation only when no more rows diverge than have a tie.

The fused kernel for attention over a pre-chosen peer set is checked the same way against the composed path it replaces, at every configuration of both pre-score sweeps and all three selectors — 60 configurations and 473,088 query rows — to a maximum deviation of 1.1×10^{-6}. In the fixed-set sweep of Section 4.4, every top-k path is checked on the timed inputs themselves, at all 48 configurations: maximum deviation 2.4×10^{-6} and no diverging row in 3.6 million. All references are the composed fp32 paths; a float64 reference is used only in the precision study, where reduced-precision rounding changes the retained set. Its timed call also converts the retained indices to the integer width the kernel reads them at, which the composed path does not need, so the figure we report for it includes a cost the comparison does not require of it.

F.2 What each implementation executes

The masked and gathered implementations compute the same function while executing different work, which is the distinction the paper turns on, so regression tests pin it. Maximum absolute deviation between the two paths is below 10^{-10} in `float64`. A FLOP counter attached to the running program reports identical totals for dense and masked — the counter scores matrix products, and masked issues the same two as dense — against a smaller total for gathered, the difference being exactly the aggregation term. Tracing every matrix product confirms that the gathered path issues no dense aggregation product and does issue the compacted one, and that every top-k variant still issues the full score product, which is assumption A2 made mechanical.

The four hand-written kernels differ only in how the retained set is reached, and each one is stated here in the terms Section 4 compares them in. All four compute the score row through the same matrix-multiply sequence as the reference, in `ieee` precision: computing it as a reduction instead changes the summation order, and a rounding-level difference in a score is enough to change which peer is retained. The per-query kernel takes k argmax rounds over the score row and reads k value rows, so its aggregation is sparse and its selection is linear in k. The threshold kernel bisects on the score's integer bit pattern for 34 steps to find the k-th largest value exactly, then weights every peer by an indicator, so its selection is independent of k and

its aggregation is dense. The sorted kernel maps each score to an order-preserving integer, packs it against its own peer index into one 64-bit word, sorts the row once, and reads the k value rows named by the top words, so it is independent of k and sparse at the same time. The pre-score kernel is given the retained indices and never forms a score row at all: one program per tile of queries walks that tile's retained peers, reading one key row and one value row per peer and accumulating with a running maximum, so nothing of size (B, N, k, d) is ever written.

F.3 Which backend serves the fused dense baseline

A probe forces each scaled-dot-product backend in turn, at both dtypes and at three agent counts, under no mask, an additive float mask and a Boolean mask, and the profiler records which kernel the default dispatcher launches. The FlashAttention backend is rejected under any non-null mask in either form, and in `fp32` under any input, so the library dense baseline runs the memory-efficient CUTLASS kernel. Forcing FlashAttention on the same unmasked `bf16` shape is faster than the kernel we do get, so a baseline able to reach it would widen rather than narrow the gap we report — which is part of why the dense baseline in this paper is the better of the library path and a kernel we wrote.

F.4 The host allocator control

Dense, masked and gathered each allocate and free several multi-MiB score matrices per forward. The default allocator returns those pages to the kernel and they are re-faulted on the next call, which at N=192 adds roughly 37,000 minor page faults per forward and inflates latency by up to 8.8×. The tax is proportional to the bytes of large temporaries each implementation churns, so it differs per implementation and does not cancel in a ratio; in an un-pinned run it inverted the ordering at one configuration. We raise the allocator's retention threshold so that freed temporaries are kept, and count minor faults per forward from the process's own resource usage across the timed region. The harness records the allocator environment, the thread counts, the affinity mask and the fault totals in each run's metadata. The fault figures quoted here come from the host microbenchmark that first exposed the effect, which predates the re-measurement campaign; the campaign's own artifacts inherit the fix and do not re-derive the fault counts, so those two numbers should be read as the diagnosis that motivated the control rather than as a released measurement.

F.5 Machine balance

Peak arithmetic rate is measured with a large matrix multiply and peak bandwidth with a large copy, in the same process as the measurements they normalise. That matters here more than it usually would: on this shared host the peak we measure varies between 58 and 77 TFLOP/s across sessions, depending on what else is running. Every fraction-of-peak figure in the paper is against the peak measured in the same process as the number it normalises, and is therefore an upper bound on the true efficiency rather than an estimate of it. Comparisons between two implementations measured in one process are unaffected; comparisons of a fraction across sections are not, and we make none. The copy source is written before it is read: an untouched allocation on this operating system is backed by the shared zero page, so a host-side copy from it re-reads one cache line and reports a bandwidth several times the machine's real figure, and an earlier version of this probe did exactly that: every CPU ridge point derived from it was too large by roughly a factor of six. Device work is also timed with device events rather than host wall-clock, since host timing around a sub-millisecond copy measures launch and synchronisation. The probe reports two ridge points per device, one with a cache-resident buffer and one with a buffer larger than the last-level cache, because the two differ by a factor of two on the host and a claim that holds for one need not hold for the other.

G Additional training analysis

G.1 Design and the registered comparisons

Twelve variants at up to twelve agents were trained. Besides the communicating and non-communicating baselines they include fixed-k attention, uniformly random peers at the same budget, an α-entmax normali-

sation (Peters et al., 2019) and a communication budget enforced by a Lagrangian (Stooke et al., 2020) on a per-agent Gumbel-softmax gate (Jang et al., 2017). Scaling in the number of agents is left to the training-free measurements of Section 4, because a training step in this pipeline is dominated by the environment (Appendix B.7). On the partially observable task the fraction of peers visible falls with the agent count while the absolute number visible rises, because the world does not grow, so no comparison of returns across agent counts is made.

The analysis plan fixes the endpoint, six comparisons and their direction, an exact two-sided permutation test, the effect sizes, a bootstrap interval flagged approximate at five seeds, and a Holm–Bonferroni correction. It also fixes the power expectation in advance: at five seeds a first rejection requires one variant's seeds to lie entirely above the other's, so any null in the family is under-powered and is not evidence of absence, and no equivalence claim is available because no equivalence margin was registered.

The primary comparison holds the communication budget, the executed aggregation cost and every hyperparameter fixed, and varies only whether peers are chosen by attention score or uniformly at random. Attention-selected minus random peers is $+4.03$, with a 95% interval of $[-3.35, +11.42]$, a standardised effect of $+0.60$ and $p = 0.357$. The point estimate favours attention by roughly two thirds of a standard deviation and the interval includes zero; the design lacked the power to resolve an effect of that size, as the registered calculation said in advance, and nothing follows about whether attention selects informatively.

G.2 Pre-registration chronology

The analysis plan was committed when eight of the forty runs in the pre-registered family had produced a final evaluation and before any of their returns had been inspected. The timestamped history in the anonymised supplementary archive establishes the ordering: the positive-control artifacts at 12:18, the plan at 12:56, and the remaining artifacts at 19:11 the same day. The plan has two commits, the second appending to its deviations log; the endpoint, comparisons, test, effect sizes, interval and correction are byte-identical to the pre-unblinding version. Timestamps establish ordering and nothing more. Data generation had demonstrably already begun.

The power statement in Appendix G.1 is arithmetic, not an estimate. With five seeds per variant an exact two-sided permutation test over $\binom{10}{5} = 252$ assignments has smallest attainable p-value $2/252 = 0.0079$, against a first Holm threshold of $0.05/6 = 0.0083$: a margin of 3.97×10^{-4}. A first rejection therefore requires one variant's five seeds to lie entirely above the other's, and any effect smaller than complete separation is undetectable by construction.

One correction is logged against the released plan. It asserts that at most one comparison in the family can reach corrected significance. That is false: Holm is a step-down procedure, so once a first rejection is obtained its later thresholds are looser and further rejections remain possible. The manuscript states the corrected version; the deviation is logged against the released plan.

G.3 The remaining comparisons

Five further comparisons were registered and all are Holm-adjusted nulls. Two deserve a note. The comparison at twelve agents has the one sign opposite to its registered direction and carries the family's largest point estimate, and we set it aside on the ground that neither variant learns the task there. That ground was not registered: it is a post-hoc exclusion of the largest observed effect, we have logged it as a deviation against the released plan, and a reader who rejects it should read the comparison as an under-powered null with a large point estimate, which is what the registered analysis returns either way. The evidence for the exclusion is this. A uniform-random policy scores -758.9 under a forty-episode evaluation, against -864.9 for the dense variant, which is worse, and -717.9 for the sparse variant, which is better but barely learning; the comparison is between a diverged run and a struggling one. The centralised critic sees the same masked observations as the actors, with no privileged global state, which departs from standard practice for this learner and is a plausible contributor. Separately, for two comparisons the bootstrap interval excludes zero while the permutation test does not. The registration named the permutation test, so it governs; the

disagreement is unsurprising at five seeds, where the interval is coarse, discrete and of uncertain coverage. We report both so that the estimated effect and the strength of evidence for it are both visible.

Descriptively, and supporting no claim: the budgeted variant meets its budget but not by the intended route, its gate collapsing to an expected support near one within the first tenth of training while the dual multiplier decays to zero; the entmax variant settles at a support of 1.64 of five peers; and attention entropy falls from 0.35 nats over five peers to 0.12 over two. The three sit on different supports and are reported only as an instrumentation check.

G.4 The partially observable environment

The variant modifies the standard cooperative task in one respect: a peer's relative-position entry is zeroed when the peer is farther than a fixed radius. Reward, action space, landmark observations, episode length and observation dimension are unchanged. Two properties limit what it can support. The mask is a zeroing rather than an explicit visibility flag, so a peer at relative position exactly zero is not distinguishable from an absent one. On a continuous position space that coincidence has probability zero and we did not observe it, but we did not instrument for it either, and the released characterisation records visible fractions only. And the visible fraction falls with the number of agents while the absolute number visible rises, because the world size is fixed, which is a difficulty confound that rules out across-agent-count return comparisons.

The communicating variants carry between 7.2% and 19.0% more trainable parameters than the communication-free baseline, since the attention module adds parameters the baseline has no analogue for. The positive controls therefore establish that the communicating architecture outperforms the baseline, not that the channel alone accounts for the difference.

H Reproduction and provenance

The measurement campaign is training-free and completes in about three hours on one device, most of it the per-format tiling searches of the precision study. `scripts/run_revision_campaign.sh` regenerates the fourteen artifacts every table and figure in this paper is built from, in the order the manuscript reports them; the equivalence sweep (`scripts/verify_all_configurations.py`), the launch-cost probe (`scripts/launch_overhead.py`), the fixed-set sweep of Section 4.4 (`scripts/bench_fixed_set.py`, about two minutes, which stores every raw sample and times each group eagerly and under CUDA-graph replay) and the backend probe of Appendix F are separate entry points and are named where they are used. Results not produced by the current campaign — the host-side allocator diagnosis and the numerical-format robustness checks — date from an earlier campaign on the same host, and we say so where they are quoted; `scripts/make_revision_tables.py` and `scripts/make_revision_figures.py` regenerate every table and figure from those artifacts, so no tabular number is transcribed by hand except those of the learning study's Table 18, which are copied from `results/reduced_design_summary.md`; `scripts/check_revision_numbers.py` re-derives the numbers that appear in prose and fails on any disagreement; and with `-verify-paper-numbers` it also checks a manifest (`scripts/paper_numbers.py`) that ties each number in the main text, in the section where it appears, to its source artifact, configuration and tolerance. The campaign runs the tiling search first, because every sweep after it re-times the dense configuration the search selects; that is what makes each ratio in this paper a ratio against the best dense implementation we have rather than against whichever one a given sweep happened to build.

The environment is one NVIDIA RTX PRO 6000 Blackwell GPU (`sm_120`) in a shared multi-GPU x86_64 server, under PyTorch 2.11 with CUDA 12.8 and Triton 3.6; the driver version and the full software environment are recorded in the supplementary archive. Every latency outside the precision study of Section 4.5 is `fp32`, timed with CUDA events around a single call, interleaved implementation by implementation and repetition by repetition, with 20 warm-up repetitions discarded, 40 to 60 timed repetitions per block, and the whole block repeated three times (the tiling search instead times each configuration 30 times after 15 warm-up calls, in one block, sequentially); the reported figure is the minimum over all samples and the spread of the three per-run medians is recorded beside it. The tiling search enumerates query tiles of 16 to 128, key tiles or head chunks of 16 to 128, two to eight warps and one to three pipeline stages, and admits a

configuration when its key, value and accumulator blocks fit a 100 kB shared-memory budget; the precision study repeats that search separately in each format. FlexAttention is called through `torch.compile` with the recompilation limit raised and fallback to the unfused path disabled, since past the default limit it silently executes an implementation that materialises the score matrix.

Three provenance caveats. The aggregate provenance record predates an upgrade of the software environment and has been corrected; the run-level provenance for the training variants lives in the per-run configuration files rather than in the aggregate record. The GPUs used for every measurement in this paper are shared with other tenants, so absolute latencies are not reproducible to better than the drift reported in Appendix E.3 and only ratios between implementations timed in the same interleaved sweep should be compared. And the activation-memory measurement is reported from a run with the card otherwise free, since a shared-host repeat exhausts memory one agent count earlier.